

STŘEDNÍ ŠKOLA TELEINFORMATIKY

OSTRAVA

příspěvková organizace



Databázové systémy

ČENĚK ĎURIAN

OSTRAVA 2025

Tento dokument byl vytvořen pro účely podpory výuky na Střední škole teleinformatiky, Ostrava, příspěvková organizace.

Název: Databázové systémy

Autor: PhDr. Čeněk Ďurian, MBA, MSc.

Studijní opora k předmětu: *Databázové systémy*

Jazyková korektura nebyla provedena, za jazykovou stránku odpovídá autor.



Toto dílo podléhá licenci:

[Creative Commons Uveďte původ-Neužívejte dílo komerčně-Zachovejte licenci 4.0
Mezinárodní Licence.](https://creativecommons.org/licenses/by-nc-sa/4.0/)

© Čeněk Ďurian

Verze dokumentu: 1.0

Obsah

1	ÚVOD	9
1.1	Data a informace	9
1.2	Historie databází.....	10
2	ZÁKLADNÍ POJMY	11
2.1	Databáze	11
2.2	Databázový systém.....	11
2.3	Entita	11
2.4	Entitní typ.....	11
2.4.1	Pravidla pro názvy entitních typů	12
2.5	Atributy	13
2.5.1	Pravidla pro tvorbu atributů	14
2.6	Propojení entitních typů, atributů a entit.....	15
3	DATABÁZOVÉ MODELY	17
3.1	Flat File model	17
3.2	Relační databáze.....	18
3.2.1	Matematický princip	18
3.2.2	Praktická aplikace	19
3.2.3	Tabulka	19
3.2.4	Relační databázová řešení.....	21
4	VZTAHY.....	22
4.1	Pod pokličkou entitních typů	22
4.2	Rekurzivní vztahy	24
4.3	Povinnost členství	26
4.3.1	Pokládání otázek k povinnosti členství.....	26
4.3.2	Grafický zápis povinnosti členství.....	28
4.4	Kardinalita vztahu	29
4.4.1	Pokládání otázek pro kardinalitu	29
4.4.2	Kardinalita typu 1:1	29
4.4.3	Kardinalita typu 1:N	30
4.4.4	Kardinalita typu M:N.....	31
4.4.5	Grafický zápis kardinality.....	32

5	KONCEPTUÁLNÍ MODEL	33
5.1	ERM a ERD	33
5.2	Symbolika kardinality	34
5.3	Příklad konceptuálního modelu.....	37
5.3.1	Rozbor povinnosti členství	37
5.3.2	Rozbor kardinality	37
5.4	ERD notace	38
6	KLÍČE.....	39
6.1	Unikátní klíč.....	39
6.2	Primární klíč.....	40
6.3	Složený klíč.....	41
6.4	Umělý klíč.....	42
6.5	Cizí klíč	43
7	LOGICKÝ MODEL.....	44
7.1	Dohodnutá konvence.....	44
7.2	Dekompozice vztahu M:N	45
7.2.1	Povinnost členství v dekomponovaném modelu	47
7.2.2	Tabulkové zobrazení.....	47
8	NORMALIZAČNÍ PROCES.....	48
8.1	První normální forma (1NF)	48
8.1.1	Úpravy v rámci jedné tabulky.....	48
8.1.2	Úpravy v rámci více tabulek.....	51
8.2	Druhá normální forma (2NF)	56
8.3	Třetí normální forma (3NF)	60
8.3.1	Změna primárního klíče.....	61
8.3.2	Další dělení tabulek	62
8.3.3	Praxe ukáže... ..	62
9	FYZICKÝ MODEL	64
9.1	Transformace na fyzický model.....	64
9.2	Charakteristiky fyzického modelu	65
9.3	Komponenty fyzického modelu	65
10	DATOVÉ TYPY V SQL SERVERU	66
10.1	Textové datové typy	66

10.2	Numerické datové typy.....	67
10.2.1	Celá čísla	67
10.2.2	Desetinná čísla	67
10.2.3	Typy pro datum a čas	68
11	ÚVOD DO JAZYKA SQL	69
11.1	Co je SQL?	69
11.2	Podkategorie SQL	70
11.3	Databázoví klienti.....	70
11.3.1	Příkazový řádek.....	70
11.3.2	Grafické uživatelské rozhraní	71
12	VYTVÁŘENÍ TABULEK – CREATE TABLE.....	72
13	ZÁKLADY OMEZENÍ	73
13.1	PRIMARY KEY	73
13.2	IDENTITY	73
13.2.1	IDENTITY se specifikací	73
13.2.2	IDENTITY bez specifikace.....	74
13.3	FOREIGN KEY	74
13.4	NOT NULL	75
13.5	UNIQUE.....	75
13.6	CHECK.....	76
13.7	DEFAULT	76
14	TVORBA TABULEK S OMEZENÍMI	77
14.1	Integritní omezení v rámci tabulky.....	77
14.2	Vytváření vazeb mezi tabulkami (FK)	78
15	ÚPRAVA TABULKY – ALTER TABLE	81
15.1	Přidání sloupce	81
15.2	Úprava existujícího sloupce	82
15.3	Vymazání sloupce	83
15.4	Přejmenování sloupce.....	84
15.5	Přejmenování tabulky	85
15.6	Vymazání tabulky.....	86
16	VKLÁDÁNÍ DAT – INSERT	87
16.1	Syntaxe	87

16.2	Vložení jednoho řádku	88
16.3	Vložení více řádků.....	88
17	ÚPRAVA DAT – UPDATE.....	90
17.1	Syntaxe	90
17.2	Aktualizace jednoho atributu.....	90
17.3	Aktualizace více atributů najednou	91
18	MAZÁNÍ DAT – DELETE	93
18.1	Syntaxe	93
18.2	Smazání jednoho řádku	93
18.3	Smazání všech řádků	93
18.4	Promazání tabulky	94
19	ZÁKLADNÍ VÝBĚR DAT – SELECT	95
19.1	Syntaxe	95
19.2	Výběr všech sloupců a všech záznamů	95
19.3	Výběr zvolených sloupců	95
19.4	Výběr konkrétních sloupců s podmínkou.....	96
20	ALIAS	97
20.1	Alias pro sloupec	97
20.2	Alias pro tabulku	98
21	LOGICKÉ OPERÁTORY	99
21.1	Operátor AND	99
21.2	Operátor OR	99
21.3	Negace	100
21.4	Práce s neznámými hodnotami	100
22	OMEZENÍ VÝSLEDKŮ PŘI VÝBĚRU	101
22.1	Filtrování pomocí WHERE	101
22.2	Omezení počtu výsledků pomocí TOP	101
22.3	Výběr z více možností pomocí IN	102
22.4	Omezení hodnot v rozsahu – BETWEEN	102
22.5	Odstranění duplicitních řádků – DISTINCT	102
23	ŘAZENÍ VÝSLEDKŮ – ORDER BY	103
23.1	Řazení podle jednoho sloupce	103
23.2	Řazení podle více sloupců	103

24	FUNKCE V SQL.....	104
24.1	Textové a znakové funkce	104
24.1.1	Délka textu – LEN	104
24.1.2	Spojení více řetězců – CONCAT	105
24.1.3	Změna velikosti písmen – LOWER a UPPER	105
24.1.4	Nahrazení textu – REPLACE.....	105
24.1.5	Výběr části textu – SUBSTRING	106
24.1.6	Část textu zleva nebo zprava LEFT a RIGHT	106
24.1.7	Odstranění mezer – TRIM, LTRIM, RTRIM	107
24.1.8	Hledání pozice znaku – CHARINDEX.....	107
24.1.9	Obrácení textu – REVERSE	108
24.2	Numerické funkce	108
24.2.1	Absolutní hodnota – ABS	108
24.2.2	Umocnění – POWER	109
24.2.3	Druhá odmocnina – SQRT	109
24.2.4	Zaokrouhlení – ROUND	109
24.2.5	Součet – SUM	110
24.2.6	Průměr – AVG	110
24.2.7	Počet – COUNT	110
24.2.8	Nejmenší a největší hodnota – MIN a MAX	111
24.3	Funkce pro datum a čas	111
24.3.1	Aktuální datum a čas – GETDATE	111
24.3.2	Získání roku, měsíce, dne – YEAR, MONTH, DAY	111
24.3.3	Část data podle volby – DATEPART	112
24.3.4	Rozdíl mezi daty – DATEDIFF	112
24.3.5	Přičítání k datu – DATEADD	112
24.3.6	Převod na typ DATE nebo TIME – ČÁST	113
24.3.7	Formátování datumu a času jako text – FORMAT	113
24.4	Ostatní užitečné funkce	114
24.4.1	Název aktuální databáze – DB_NAME	114
24.4.2	ID posledního vloženého záznamu – SCOPE_IDENTITY	114
24.4.3	Uživatelské jméno – SYSTEM_USER.....	114
25	SESKUPENÍ VÝSLEDKŮ – GROUP BY	115

25.1	Syntaxe	115
25.2	Příklady.....	116
26	SPOJOVÁNÍ TABULEK	118
26.1	Vnitřní spojení	118
26.2	Vnější spojení	120
26.2.1	LEFT JOIN.....	120
26.2.2	RIGHT JOIN	122
26.2.3	FULL JOIN	123
27	OPRÁVNĚNÍ.....	126
27.1	LOGIN.....	126
27.2	USER.....	127
27.3	GRANT	128
27.3.1	Udělení jednoho oprávnění	128
27.3.2	Udělení více oprávnění najednou.....	129
27.3.3	Udělení oprávnění na konkrétní sloupce.....	129
27.4	REVOKE.....	130
27.4.1	Odebrání jednoho oprávnění	130
27.4.2	Odebrání více oprávnění najednou.....	130
27.4.3	Odebrání oprávnění na konkrétní sloupce.....	130
27.5	Odebrání uživatele a přihlášení	131
27.5.1	Odstranění uživatele z databáze	131
27.5.2	Odstranění loginu ze serveru.....	131

1 Úvod

Snad většina typů informací, jako jsou třeba smsky, příspěvky na sociálních médiích, nákupy v e-shopech, stahovaná hudba či videa a mnoho dalšího, to vše je uloženo v databázích, s nimiž jsme ve spojení každý den, ať už si to uvědomujeme či nikoliv.

1.1 Data a informace

Slova **data** a **informace** jsou často považována za synonyma, často se zaměňují, někteří lidé si myslí, že mezi nimi není žádný rozdíl. Nicméně, tato slova mají odlišný význam.

- Za data považujeme **surový nezpracovaný materiál bez kontextu**, který sám o sobě může vypadat pro člověka poněkud vytržený z kontextu.
- Za informace považujeme konkrétní údaje, které mají **význam**, dávají nám nějaký smysl.

Uveďme si jednoduchý příklad:

Data

Pondělí	22
Úterý	23
Středa	24
Čtvrtek	25
Pátek	26
Sobota	27
Neděle	23

Informace

Průměrná teplota během týdne byla 24,3 °C.
Nejchladnější den byl v pondělí s teplotou 22 °C.
Naopak nejteplejší den byl v sobotu s teplotou 27 °C.
Teplota během týdne postupně stoupala pak v neděli klesla.

Vidíme, že když surovým datům přidáme trošku vysvětlení (kontextu), začínají nám dávat smysl a můžeme hovořit o **informacích**.

1.2 Historie databází

1960	V období let 1960 se začíná soukromým firmám vyplácet pořizování počítačů.
1970-1972	Matematik Edgar Frank Codd navrhuje relační model databází, čímž je umožněno odpojení logické organizace dat od fyzického úložiště.
1976	Počítačový vědec Peter Chen navrhuje Entity Relationship Model (ERM) pro databázový návrh.
1980	V období kolem roku 1980 přichází na scénu jazyk SQL (structured Query Language), který se stává zavedeným standardem.
1990	Startuje velký boom Internetu a začíná se pracovat na vývoji tzv. connectorů, které by propojily Web, Internet a Databáze.
2000	Databázový vývoj pokračuje, příkladem jsou komerční weby Yahoo, Google, Amazon, státní systémy, nemocniční a školské systémy atd.
2010	Okolo roku 2010 začíná nová éra, založená na tzv. cloud computingu, což je poskytování služeb skrz cloud (internet), který je nabízen např. společnostmi jako jsou Microsoft, Oracle, Apple a další.

2 Základní Pojmy

2.1 Databáze

Co je vlastně databáze? Můžeme říci, že se jedná o **centralizovanou a strukturovanou** sadu dat, která jsou uložena v počítačovém systému. To nám umožňuje s daty provádět různé operace, např. vkládání dat do databáze, získávání dat z databáze, jejich modifikování (úpravu) nebo mazání. Databáze je obvykle spravována Databázovým Administrátorem (tzv. DBA).

2.2 Databázový systém

O data a mnohem více se stará specializovaný software, kterému se říká anglicky **Database Management System (DBMS)**. Tento pojem má i český ekvivalent, tzv. Systém Řízení Báze Dat (SŘBD). My si vystačíme s názvem anglickým.

O DBMS bychom měli vědět, že je to specializovaný software, který se stará např. o následující:

- kontrola úložiště
- organizace a získávání dat
- řízení uživatelského přístupu

2.3 Entita

Je dost možné, že tento pojem bude pro mnohé zcela nový a možná poněkud zvláštní. Ve filozofické rovině je možné slovo "entita" definovat jako "jsoucno", tedy něco, co prostě **je** či **existuje**. Když na tento pojem půjdeme trošku jednodušeji, můžeme říci, že entita je něco, cokoliv, například osoba, zvíře, věc, událost či proces.

V informatice, především v databázových systémech, považujeme za entitu **jakýkoliv objekt nebo věc v reálném světě**, o které potřebujeme uchovávat data. Může to být například osoba, zvíře, věc, proces (například objednávka nebo rezervace), nebo událost. Tyto entity mají určité vlastnosti (atributy) a mohou mít vztahy k jiným entitám.

2.4 Entitní typ

Entitní typ je pojmenování pro **kategorii entit**, které mají podobné, resp. shodné vlastnosti. Pokud zamýšlíme, že budeme v našem systému chtít zpracovávat např. studenty (student jako entita reálného

světa), v rámci databázového systému pro ně vytváříme tzv. entitní typ s názvem **Student**. U tohoto entitního typu následně definujeme námi požadované vlastnosti. Například, že každá entita Student by měla mít nějaké jméno, příjmení, datum narození apod. Tyto vlastnosti pak budou platit pro každou entitu tohoto entitního typu (kategorie). Ukažme si příklad.

Příklad

Zákazník, který je manažerem hotelového řetězce nám sdělil, že potřebuje vytvořit systém, který by evidoval informace o jeho **hotelích, návštěvnících a pokojích**. Potřebuje také, aby systém uměl zpracovávat **rezervace** návštěvníků hotelu. Na základě těchto informací víme, že do procesu vývoje databáze budou vstupovat určité 4 entitní typy.

Hotel

Navstevnik

Pokoj

Rezervace

Všimněme si, že zobrazené entitní typy jsou opravdu různé. Máme zde entitní typ *Návštěvník*, což symbolizuje nějakého člověka, dále *Hotel* a *Pokoj*, což bychom mohli zařadit do kategorie věcí, a pak je tu *Rezervace*, která by se dala charakterizovat jako proces. Entitou, pro níž popisujeme entitní typ může být zkrátka cokoliv, co potřebujeme nějak evidovat a zpracovávat.

2.4.1 Pravidla pro názvy entitních typů

Pro pojmenovávání entitních typů existují určitá pravidla. Neexistuje však žádné univerzální pravidlo. V každém prostředí mohou být doporučena jiná pravidla. Například pro práci s databázovým systémem MySQL je doporučeno psát názvy entitních typů pomocí notace „**snake_case**“. V případě databázových systémů Microsoft SQL Server je zase využívána notace „**PascalCase**“. V ostatních systémech tomu může být jinak. Co je však pro pojmenovávání entit doporučeno vždy, je následující:

- Název entity je podstatné jméno (*popř. v kombinaci s přídavným jménem*)
- Používáme jednotné číslo
- Bez české diakritiky
- Bez mezer
- Bez speciálních znaků
- Bez čísel (je-li to možné)

Pojďme se podívat, jak vypadá správný zápis názvů entitních typů v obou zmíněných notacích:

Originální název	PascalCase	snake_case
Zaměstnanec	Zamestnanec	zamestnanec
Operační systém	OperacniSystem	operacni_system
Hotelový pokoj	HotelovyPokoj	hotelovy_pokoj
Počítač	Pocitac	pocitac
Kniha	Kniha	kniha

Snad jen pro úplnost dodejme, že notace **PascalCase** je způsob zápisu, kdy každé slovo začíná velkým písmenem a mezery jsou odstraněny. U notace **snake_case** jsou všechna písmena malá a v případě víceslovných názvů se mezera nahrazuje podtržítkem.



V tomto předmětu budeme dodržovat notaci **PascalCase**.

2.5 Atributy

Slovo **atribut** můžeme nahradit také slovem **vlastnost**. V našem kontextu uvažujeme atributy, které se pojí s entitními typy. Každý entitní typ má totiž určitý počet atributů neboli vlastností, které lze chápat jako informace o entitách. Zvolme si třeba entitu *Navstevnik* a zamysleme se, jaké informace o návštěvnících bychom mohli zpracovávat. Řekněme, že nám bude pro začátek stačit jméno, příjmení a rodné číslo.

Jmeno	Prijmeni	RodneCislo
Jan	Černý	101092/0523
Irena	Bílá	035608/2336
Petr	Novák	010485/6841

Z výše uvedené ukázky vidíme entitní typ symbolizovaný pomocí tabulky s názvem **Navstevnik**, která zahrnuje 3 atributy **Jmeno**, **Prijmeni** a **RodneCislo**.

2.5.1 Pravidla pro tvorbu atributů

Stejně jako v případě samotných entitních typů, také u jejich atributů platí určitá pravidla pro pojmenování. Opět se dá říci, že univerzální pravidlo neexistuje a zase vždy záleží, v jakém prostředí a s jakým databázovým systémem pracujeme. I v případě atributů dodržujeme v rámci tohoto předmětu notaci **PascalCase**. Ostatní pravidla jsou následující:

- podstatné jméno
- jednotné číslo
- bez mezer
- bez české diakritiky
- bez speciálních znaků
- bez čísel (je-li to možné)
- název atributu **odlišný od názvu entitního typu**, pod který spadá

Všimněme si, že pravidla pro tvorbu entitních typů a atributů se netýkají uživatelských dat! Zatímco názvy entitních typů a atributů píšeme dle výše uvedených pravidel, uživatelská data píšeme v běžném přirozeném jazyce. Je to proto, že názvy entitních typů, atributů a později i dalších součástí databázového systému považujeme za **technická data**, s nimiž přicházejí do styku **vývojáři** a další IT specialisté, kteří mají mezi sebou dohodu o zapisování názvů apod. (tzv. konvence).

Oproti tomu **uživatelská data** jsou vidět v hotovém produktu. Jinými slovy je vidí **koncoví uživatelé** na svých obrazovkách... a ti je chtějí vidět v přirozené podobě. Jistě by se panu Pavlu Novákovi nelíbilo, kdyby ho námi vyvinutý systém zobrazoval jako PavelNovak nebo by město Český Těšín bylo zobrazeno jako CeskyTessin.

Technická data	Uživatelská data
Struktura databáze (entity, atributy...)	Výsledné informace v aplikaci
Programování (proměnné, příkazy...)	Výsledky výpočtů na obrazovce
Běžný uživatel je nevidí – jsou schovaná	Pracuje s nimi koncový uživatel
Pracují s nimi vývojáři	

2.6 Propojení entitních typů, atributů a entit

Probrali jsme společně tři základní prvky databázového modelování – entity, entitní typ a atributy. Nyní je čas, abychom se podívali, jak tyto prvky spolu souvisejí:

1. **Entitní typ** představuje kategorii entit
2. V entitním typu jsou definovány **atributy**, které říkají, jaká data budeme o entitách zpracovávat
3. Na základě vytvoření entitního typu a definování atributů mohou vznikat jednotlivé **entity**.

Zkusme si ukázat příklad, kdy pro obchodní řetězec máme zpracovat část systému, v němž bude evidováno ovoce.

Na základě informací od zákazníka jsme stanovili entitní typ Ovoce. Zákazník nám také prozradil, že potřebují o ovoci evidovat název, barvu a cenu. Entitnímu typu Ovoce jsme tedy definovali tři atributy: **Název**, **Barva** a **Cena**. Takže víme, že každá entita tohoto entitního typu bude mít nějaký název, barvu a taky nějakou tu cenu. Pro účely ukázky se dohodněme, že cena bude v Kč za 1 kg váhy ovoce.

Název	Barva	Cena
Jablko	Červená	40
Hruška	Zelená	50
Banán	Žlutá	35
Malina	Růžová	200
Borůvka	Fialová	300

Když se podíváme na tabulku, můžeme říci, že entitní typ **Ovoce** má **3 atributy** a obsahuje celkem **5 entit**:

1. Červené jablko s cenou 40 Kč za 1 kg.
2. Zelená hruška s cenou 50 Kč za 1 kg.
3. Žlutý banán s cenou 35 Kč za 1 kg.
4. Růžová malina s cenou 200 Kč za 1 kg.
5. Fialová borůvka s cenou 300 Kč za 1 kg.

Pojďme zkusit ještě jeden příklad. Zákazník, který je majitelem autoservisu potřebuje nový systém, který by, mimo jiné, evidoval značku, model, SPZ a typ paliva všech servisovaných automobilů.

Na základě získaných informací stanovujeme entitní typ Automobil, v němž definujeme atributy Značka, Model a Spz.

Značka	Model	Spz	TypPaliva
Škoda	Octavia	1AA 1234	benzín
Volkswagen	Passat	2BB 5678	nafta
Ford	Focus	EL 123AB	elektřina
Toyota	Corolla	4DD 3456	benzín
BMW	X5	EL 359BA	elektřina

Poněvadž jsme již ostřílení odborníci, můžeme naším zkušeným okem zhodnotit tabulku a prohlásit, že vidíme entitní typ **Automobil** se čtyřmi definovanými **atributy** a pěti **entitami**.

3 Databázové modely

V této kapitole si ukážeme základní typy databázových modelů. Databázový model je teoretický koncept, který nám říká, jakým způsobem budou data v databázi organizována, strukturována, případně propojena.

Za základní databázové modely můžeme považovat následující:

- **Flat File model**
- **Relační model**
- Objektově orientovaný model
- Hierarchický model
- Síťový model

Z výše uvedených modelů si vysvětlíme dva, s nimiž rozhodně přijdeme do styku. Jedná se o **Flat File model**, který budeme občas využívat, a dále o **Relační model**, na nějž je tento předmět zaměřen.

3.1 Flat File model

Flat File modely jsou typicky tvořeny jednou (pomyslnou) tabulkou v podobě **prostého textu**. Každý řádek textu symbolizuje jednu entitu. Každý řádek může být složen z více pomyslných sloupců, přičemž tyto sloupce a jejich hodnoty jsou mezi sebou odděleny tzv. separátorem. Jako separátor se nejčastěji používá **čárka** nebo **středník**. Pomocí separátoru zjistíme my i počítač, kde jeden sloupec končí a kde začíná druhý.

Typickým příkladem Flat File modelu jsou **soubory CSV** (Comma-separated values). Jedná se o obyčejný textový soubor s příponou .csv, v němž jsou data zapsána přesně tak, jak jsme si pospali výše. Můžeme jej vytvořit například v aplikaci **Poznámkový blok** nebo **Excel**.

Zkusme si představit entitní typ *Zamestnanec*, který bude mít definovány atributy *CisloZamestnance*, *Jmeno* a *Prijmeni*. Uvnitř CSV souboru by instance tohoto entitního typu vypadaly asi nějak takto:

Zamestnanec.csv

```
CisloZamestnance,Jmeno,Prijmeni
6541,Jan,Novák
6521,Karel,Vomáčka
6214,Lucie,Stará
6952,Růžena,Vostrá
```

Jednotlivé hodnoty oněch pomyslných sloupců jsou odděleny separátorem, v tomto konkrétním případě je jím čárka. Soubor by pak mohl ideálně nést název *Zamestnanec.csv* či *zamestnanec.csv*.

Pojďme nyní terminologicky uchopit tuto ukázkou. Soubor CSV tedy představuje **entitní typ**. Jednotlivé čárkou oddělené hodnoty prvního řádku reprezentují **atributy** entitního typu. A další jednotlivé řádky představují **entity** tohoto entitního typu.

Výhody Flat File:

- Jednoduché pochopit
- Jednoduché vytvořit
- Snadné získání dat

Nevýhody Flat File:

- Slabé zabezpečení
- Nekonzistentnost dat
- Redundance (nadbytečnost) dat
- Pomalé v případě většího množství záznamů či souborů
- Data nejsou mezi sebou propojena

3.2 Relační databáze

Relační databáze je typ databáze, který souvisí s relačním databázovým modelem, jak jej definoval Edgar F. Codd a využívá konceptu relace z matematiky k organizaci dat.

3.2.1 Matematický princip

Relace je v matematice podmnožina kartézského součinu dvou nebo více množin.

Pokud máme například množiny A a B , pak relace R mezi těmito množinami je množina uspořádaných dvojic (a, b) , kde $a \in A$ a $b \in B$.

Mějme množiny:

- $A = \{1, 2, 3\}$
- $B = \{x, y\}$

Relace R může být definována jako $R = \{ (1, x), (2, y), (3, x) \}$.

Relační schéma je formální popis struktury relační databáze. Definuje, jak jsou data organizována do tabulek (relací) a jaké jsou vztahy mezi těmito tabulkami. Každá tabulka má svůj název a je složena z atributů (sloupců).

Relační schéma R je výraz ve tvaru $R(A, f)$, kde R je jménem schématu, $A = \{A_1, A_2, \dots, A_n\}$ je konečnou množinou jmen atributů, a f je zobrazení přiřazující každému jménu atributu A_i neprázdnou množinu D_i přípustných hodnot, kterou nazýváme doménou atributu, tj. $f(A_i) = D_i$.



V oboru **informatiky** jsou n -tice běžně zapisovány např. takto: $\{(1, x), (2, y), (3, x)\}$.
V oboru **matematiky** n -tice zapisujeme s hranatými závorkami: $\{[1, x], [2, y], [3, x]\}$.

3.2.2 Praktická aplikace

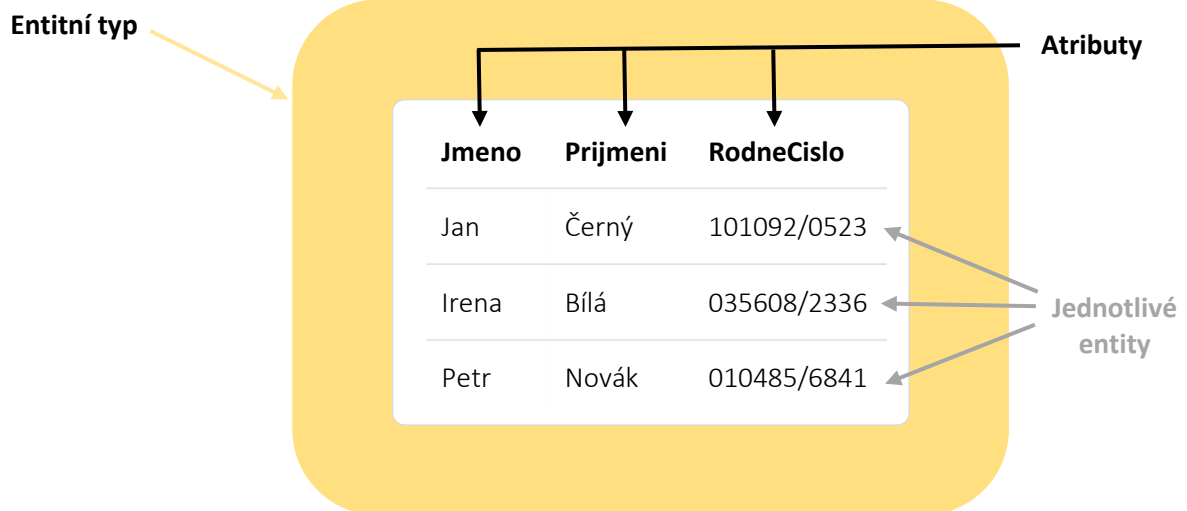
Relací je myšlena množina uspořádaných n -tic, přičemž v relačních databázích je to právě **tabulka**, kde každý **řádek** (entita) představuje jednu **n -tici**.

Matematické principy jsou neopomenutelné, nicméně v rámci tohoto předmětu nám bude stačit, když si zapamatujeme, že **relační databáze vycházejí z matematického modelu**, data v nich jsou uložena v podobě **tabulek** a jednotlivé tabulky mohou mít (a obvykle mají) mezi sebou **vztahy**.

3.2.3 Tabulka

Entitní typy jsou v relačních databázích symbolizovány **tabulkami**. Každá tabulka v relační databázi je složena ze sloupců a řádků:

- **Sloupce** – hrají roli atributů
- **Řádky** – představují jednotlivé instance (výskyty) entitního typu... co řádek, to instance. V rámci tabulek se instancím často říká záznamy.



Takže si to shrňme. Entitní typ je v relačních databázích symbolizován tabulkou. Atributy jsou symbolizovány sloupce. Instance jsou symbolizovány řádky (záznamy). Entitním typem je tedy celá tabulka, se vším, co k ní patří.

Výhody relačních databází:

- Nižší redundance dat (nadbytečnost)
- Předcházení nekonzistentnosti
- Výkonnost
- Integrita dat (celistvost)
- Bezpečnost dat

Pravidla pro tabulky v relačních databázích:

- Každá tabulka v jedné databázi musí mít odlišný název
- Každá tabulka může obsahovat více sloupců i řádků
- Každý sloupec v tabulce musí mít odlišný název
- V rámci jednoho sloupce jsou na každém řádku hodnoty stejného datového typu



Pokud některým pojmům ve výhodách nebo pravidlech nerozumíš, vůbec nic se neděje, vše si postupně vysvětlíme, až nadejde správný čas.

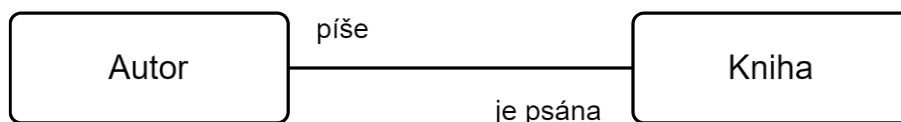
3.2.4 Relační databázová řešení

Technologická řešení v podobě relačních databázových systémů nabízí více poskytovatelů. Každé řešení má určité výhody a nevýhody. V každém se pracuje trochu odlišně, avšak základní principy pro relační databáze platí pro všechny. Mezi těmi známějšími relačními databázovými systémy jsou Microsoft SQL Server, Oracle Database, MySQL či PostgreSQL. Ukažme si základní srovnání těchto produktů.

			
Microsoft SQL Server	Oracle Database	MySQL	PostgreSQL
Komerční	Komerční	Open Source	Open Source
Vysoký výkon	Vysoký výkon	Dobrý výkon	Vysoký výkon
Komerční podpora	Komerční podpora	Velká komunita	Velká komunita
Windows, Linux	Windows, Linux, Unix	Windows, Linux, macOS	Windows, Linux, macOS
odkaz na web	odkaz na web	odkaz na web	odkaz na web

4 Vztahy

Pojmem „vztah“ jsou v oblasti databázových systémů myšleny **asociace mezi jednotlivými instancemi entitních typů**. V rámci diagramů jsou vztahy zakresleny jako čáry mezi entitními typy. Na každém konci této čáry je uvedeno pojmenování tohoto vztahu, které pomáhá vztah lépe pochopit. Popis vztahu je psán **v přítomném čase**. Pojďme se podívat na ukázkou.



V diagramu vidíme dva entitní typy, **Autor** a **Kniha**, jejichž instance mají mezi sebou vztah. Ten je zakreslen pomocí čáry mezi oběma entitními typy. Na obou koncích čáry je zapsáno pojmenování tohoto vztahu. Všimněme si, že pojmenování je vždy zapisováno jakoby očima daného entitního typu či jeho jednotlivých instancí.

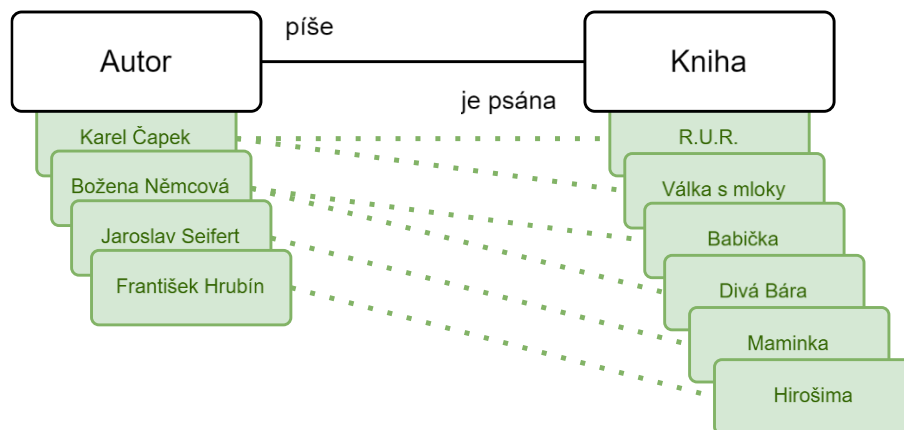
- Když začneme číst nejprve zleva doprava, čteme následovně: **autor píše knihu**.
- Pak čteme zprava doleva: **kniha je psána autorem**.

Pojmenování vztahu, které je umístěno blíže danému entitnímu typu, mu patří. Daný vztah zabezpečuje jednu činnost (psaní knihy autorem), ale pro každý entitní typ tento vztah popisujeme „jeho očima“, aby bylo vše na první pohled jasné, proto dva názvy pro jeden vztah... prostě, tak nějak pro jistotu. Taková je konvence, kterou budeme dodržovat.

4.1 Pod pokličkou entitních typů

Je velice důležité si uvědomit, že vztah je sice nakreslen mezi entitními typy, ale skutečné vztahy mají mezi sebou **instance daného entitního typu** (tedy jednotlivé entity uvnitř). Pojďme se blíže podívat na ukázkou s autorem a knihou. Jak bylo již popisováno, entitní typ je kategorie jednotlivých entit (instancí), které jsou jakoby uvnitř tohoto entitního typu.

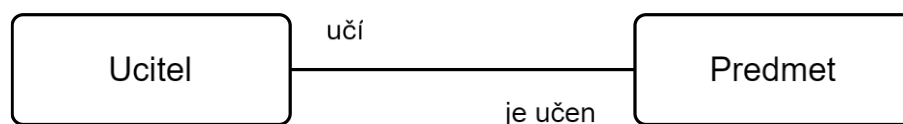
Pokud máme entitní typ Autor, představme si, že tato kategorie obsahuje konkrétní autory, např. „Karla Čapka“, „Boženu Němcovou“ atd. Podobně entitní typ Kniha obsahuje konkrétní knihy, jako jsou „Babička“, „Válka s mloky“ apod. Skutečné vztahy (například autor napsal knihu) existují mezi jednotlivými instancemi těchto entitních typů. Např. Karel Čapek (instance entitního typu Autor) napsal "Válku s mloky" (instance entitního typu Kniha).



Když pak pojmenováváme vztahy entitních typů, stále myslíme na to, že tyto **vztahy mají** mezi sebou **instance uvnitř** těchto entitních typů. Nějaký konkrétní autor píše nějakou jednu nebo více konkrétních knih. A naopak, nějaká konkrétní kniha je tím pádem psána některým z konkrétních autorů.

Takže vztahy na diagramu zakreslujeme sice mezi entitními typy, ale v konečném výsledku budou realizovány na jednotlivých instancích uvnitř entitních typů.

Pojďme si vyzkoušet další příklad:



V tomto diagramu vidíme dva entitní typy, **Ucitel** a **Predmet**. Z ukázky je patrné, že se jedná asi o vývoj nějakého školního informačního systému, v němž budou tyto entitní typy figurovat. Diagram kromě dvou entitních typů popisuje také jejich vztah.

- Když budeme opět číst zleva doprava, bude vztah (očima učitele) následující: **učitel učí předmět**.
- Když začneme číst v opačném směru, (očima předmětu), pak čteme: **předmět je učen učitelem**.

Konkrétním příkladem by pak mohlo být následující:

Čeněk Ďurian (instance entitního typu **Ucitel**) učí Databázové systémy (instance entitního typu **Predmet**) a také i z druhého konce... Databázové systémy (instance entitního typu **Predmet**) je učen Čeňkem Ďurianem (instance entitního typu **Ucitel**).

A takových vztahů mezi instancemi může být v rámci propojení dvou entitních typů mnoho.

Protože příkladů není nikdy dost, pojďme si uvést ještě jeden:

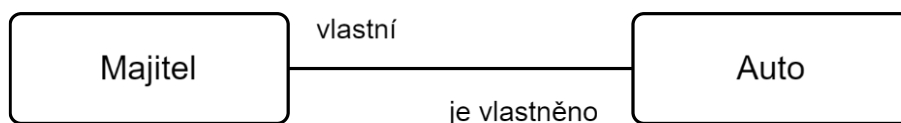


Diagram popisuje entitní typy **Majitel** a **Auto**, a jejich vzájemný vztah. Aby byl vztah na první pohled pochopitelný nejen pro nás, ale také např. pro další členy vývojového týmu, opět nebude chybět popis vztahu z pohledu obou entitních typů.

- Pojďme číst zleva doprava: **majitel vlastní auto**.
- Pojďme číst zprava doleva: **auto je vlastněno majitelem**.

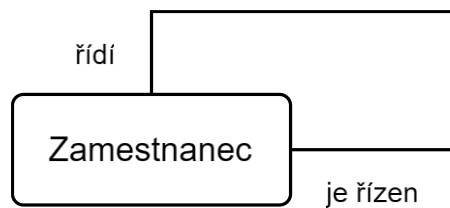
4.2 Rekurzivní vztahy

V praxi se používá i jeden možná zdánlivě zvláštní typ vztahu, ale dává smysl a je praktický. Je to případ, kdy instance mohou být ve vztahu s jinými instancemi téhož entitního typu. Jinými slovy, entitní typ může mít zakreslen vztah sám se sebou. Typickým příkladem by mohl být entitní typ **Zamestnanec**, který bude mít atributy **CisloZamestnance**, **Jmeno**, **Prijmeni** a **Nadrizeny**. Pojďme se podívat, jak by vypadala tabulka.

CisloZamestnance	Jmeno	Prijmeni	Nadrizeny
001	Petr	Nový	002
002	Ivona	Modrá	003
003	Jan	Černý	

Každý zaměstnanec v našem systému bude mít přiděleného nějakého nadřízeného. Když se zamyslíme, přijdeme na to, že nadřízený je vlastně taky nějakým zaměstnancem. Takže každý zaměstnanec může mít přiděleného jiného zaměstnance jako svého šéfa.

No, a v případě Jana Černého uvádět nadřízeného nebudeme, protože se jedná o majitele společnosti, který už žádného nadřízeného nemá. Případně bychom to mohli provést i tak, že by jeho nadřízeným byl on sám, tedy zaměstnanec s číslem 003 bude mít nadřízeného šéfa taktéž zaměstnance s číslem 003.



Tento model je speciálním případem, takže asi není divu, že i čára symbolizující vztah bude vypadat trošku jinak. Pokud mají mezi sebou vztahy instance téhož entitního typu, pak bude u entitního typu jeden konec čáry začínat a druhý také končit. Když opět začneme tento vztah číst z jedné strany a pak naopak, bude to vypadat následovně:

- Zaměstnanec (jako šéf) řídí zaměstnance
- Zaměstnanec je řízen zaměstnancem (jako šéfem)

Pojďme si ještě vyzkoušet správnou **terminologii** na vybrané ukázce, k níž si vypůjčíme opět příklad s entitním typem Zamestnanec.

CisloZamestnance	Jmeno	Prijmeni	Nadrizeny
001	Petr	Nový	002
002	Ivona	Modrá	003
003	Jan	Černý	

Terminologie

V rámci databázového systému jsme se rozhodli, že budeme zpracovávat informace o zaměstnancích jakožto o objektech reálného světa (entitách). Pro tento účel jsme vytvořili entitní typ **Zamestnanec**, u něhož jsme nadefinovali potřebné atributy. Entitní typ Zamestnanec má tedy nadefinovány **4 atributy**: CisloZamestnance, Jmeno, Prijmeni a Nadrizeny a obsahuje celkem **3 instance**. Instance entitního typu Zamestnanec, která má v atributu CisloZamestnance hodnotu 001 má vztah s instancí Entitního typu Zamestnanec, která má v atributu CisloZamestnance hodnotu 002. Instance entitního typu Zamestnanec, která má v atributu CisloZamestnance hodnotu 002 má vztah s instancí entitního typu Zamestnanec, která má v atributu CisloZamestnance hodnotu 003.

4.3 Povinnost členství

V předchozím textu jsme si vysvětlovali vztahy mezi entitními typy. Každý entitní typ byl součástí společného vztahu. Například **autor**, který psal **knihu**. A samozřejmě, abychom nezapomněli i na pohled očima druhé strany, **kniha**, která byla napsána **autorem**. Nyní se zaměříme na povinnost členství v těchto vztazích, která **určuje minimální počet vztahů instancí, kterých se entity účastní**. Členství entity ve vztahu je buď **povinné** (Mandatory) nebo **volitelné** (Optional).

4.3.1 Pokládání otázek k povinnosti členství

Během výuky mých studentů se osvědčilo zaměřit se na jeden konkrétní styl pokládání otázek, který funguje. Pokud chceme zjistit, zda má entitní typ povinnost členství ve vztahu k dalšímu entitnímu typu, pokládáme otázky způsobem, jako bychom se chtěli ujistit, **zda je to opravdu nutné**.

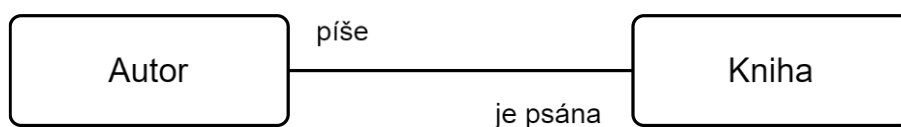
Musí každá instance entitního typu mít vztah s některou instancí jiného entitního typu?

- **Musí?** Pak je to **povinné**.
- **Nemusí?** Pak je to **volitelné**.



Otázky v rámci povinnosti směřujeme k **instancím** entitních typů.

Pojďme to společně protrénovat. Vezměme si dřívější příklad mezi autorem a knihou.



Do společného vztahu vstupují entitní typy Autor a Kniha. Nás bude nyní zajímat, zda je členství ve vztahu povinné či volitelné. Tuto otázku zkoumáme opět vždy **pro obě strany vztahu**, tedy v tomto případě od autora ke knize a naopak.

Musí každý autor (každá instance) napsat nějakou knihu?

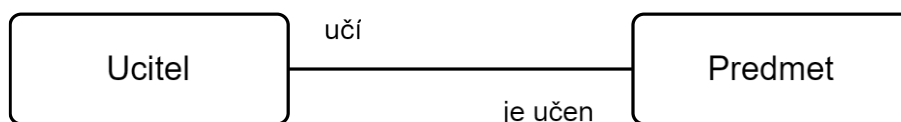
- Pokud **musí každý autor** napsat nějakou knihu, pak je členství entitního typu Autor ve vztahu k entitnímu typu Kniha **povinné**.
- Pokud je naopak možná situace, kdy **nemusí každý autor** nutně napsat nějakou knihu, pak je členství entitního typu Autor ve vztahu k entitnímu typu Kniha **volitelné**.

Musí každá kniha být napsána nějakým autorem?

- Pokud **musí každá kniha** být někým napsána, pak je členství entitního typu Kniha ve vztahu k entitnímu typu Autor **povinné**.
- Naopak, pokud **každá kniha nemusí** být nutně někým napsána (smí být evidována v systému i bez autora), je členství entitního typu Kniha v tomto vztahu nepovinné, tedy **volitelné**.

Mějme na paměti, že nikdy **neexistuje univerzální odpověď**. Každý entitní typ, jeho atributy i vztahy **závisí na konkrétní situaci**. Pokud vytváříme systém pro zákazníka, musí nám právě zákazník odpovědět na tyto otázky. Bude chtít, aby budoucí systém umožňoval evidovat knihy bez autorů nebo evidovat autory bez knih? Nebo si zákazník bude přát, aby nebylo možno do systému zapsat autora, který by nenapsal žádnou knihu? Toto vše je součástí analýzy potřeb zákazníka, kterého musíme vždy vyzpovídat.

Zkusme si ještě jeden příklad s učitelem a předmětem.



Opět se budeme ptát... **musí každý učitel** učit nějaký předmět? Nebo může existovat i učitel, který by nic neučil? Normálně bychom si asi řekli, že každý učitel by měl něco učit. Jenže co když je učitel veden v systému, ale je dlouhodobě nemocen? Nebo je paní učitelka na mateřské dovolené s miminkem? Pak tento učitel v databázi existuje, ale neučí žádný předmět. Kdyby nám zákazník řekl takovéto informace, dospěli bychom k závěru, že členství entitního typu Ucitel ve vztahu k entitnímu typu Predmet je **volitelné**.

A co předmět? **Musí každý předmět** být někým učen? K čemu by byl předmět bez učitele? Zde velmi záleží na konkrétní situaci. Může se totiž stát, že ve škole existuje volitelný předmět, na který se nikdo z žáků nepřihlásil, proto nebude v daném školním roce nikým učen, ale v systému bude stále evidován. To by znamenalo, že členství entitního typu Predmet ve vztahu k entitnímu typu Ucitel by bylo **volitelné**.

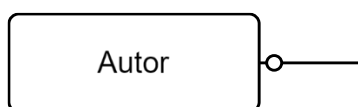
Nebo se od zákazníka dozvíme, že všechny předměty jsou v jejich škole povinné, takže k takové situaci určitě nikdy nedojde. V tom případě by bylo členství entitního typu Predmet ve vztahu **povinné**. Vidíte, jak se může situace na základě získaných informací měnit?

4.3.2 Grafický zápis povinnosti členství

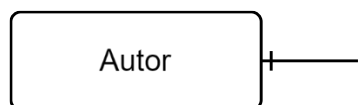
Nyní se podíváme, jak se zapisuje povinnost, resp. volitelnost v rámci vztahů mezi entitními typy v diagramech.

- Pokud je členství ve vztahu **povinné**, pak jej zakreslujeme pomocí **svislé čárky** (jakoby jedničky).
- Pokud je členství ve vztahu **volitelné**, pak jej zapisujeme pomocí **prázdného kolečka** (jakoby nuly).

Pokud bychom dospěli k závěru, že členství entitního typu Autor ve vztahu je **volitelné**, pak bychom v diagramu na konci čáry u entitního typu Autor zakreslili **prázdné kolečko**.



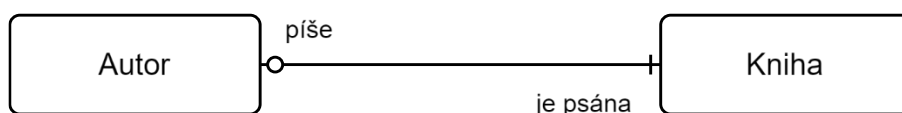
V případě, že by mělo být členství entitního typu Autor ve vztahu **povinné**, pak bychom jej v diagramu zakreslili čárkou nakreslenou kolmo k čáře znázorňující vztah.



Zbývá ještě ukázat, jak by vypadalo kompletní zakreslení povinnosti členství pro vztah obou entitních typů na diagramu. Řekněme, že v naší databázi bude situace následující:

- Každý autor **nemusí** mít napsanou nějakou knihu
- Každá kniha však **musí** být napsána nějakým autorem

Z tohoto plyne, že členství ve vztahu pro entitní typ **Autor** je **nepovinné**, a členství ve vztahu pro entitní typ **Kniha** je **povinné**. Diagram by mohl vypadat nějak takto:



4.4 Kardinalita vztahu

V této části se dostáváme k finální fázi vztahů. K identifikaci vztahů a jejich povinnosti nám nyní přibývá ještě tzv. kardinalita vztahu, s níž souvisí **další dva symboly**, které jsou zobrazovány na koncích čar v diagramech. Slovo kardinalita můžeme také vyjádřit pojmem „mohutnost“, tedy „jak moc“. Kardinalita vztahu **určuje maximální počet vztahů instancí, kterých se entita účastní**. V tomto předmětu budeme řešit hlavně binární typy kardinality **1:1**, **1:N** a **M:N**.

4.4.1 Pokládání otázek pro kardinalitu

U povinnosti členství jsme si řekli, jak správným a bezpečným způsobem pokládat otázky, abychom zjistili povinnost členství mezi entitními typy, resp. jejich instancemi. Stejně důležité je umět pokládat otázky (ať už sobě či zákazníkovi) pro zjištění kardinality.

Zatímco na povinnost se ptáme jakoby vyhýbavým způsobem, zda **musí**, zda je to vlastně vůbec **nutné**, u zjišťování typu kardinality se snažíme zjistit, jak moc vztahů je možno mezi instancemi navázat a ptáme se naopak, zda **může**, zda je to **možné**.

Může libovolná instance entitního typu mít **víc než jeden** vztah s některou instancí jiného entitního typu?

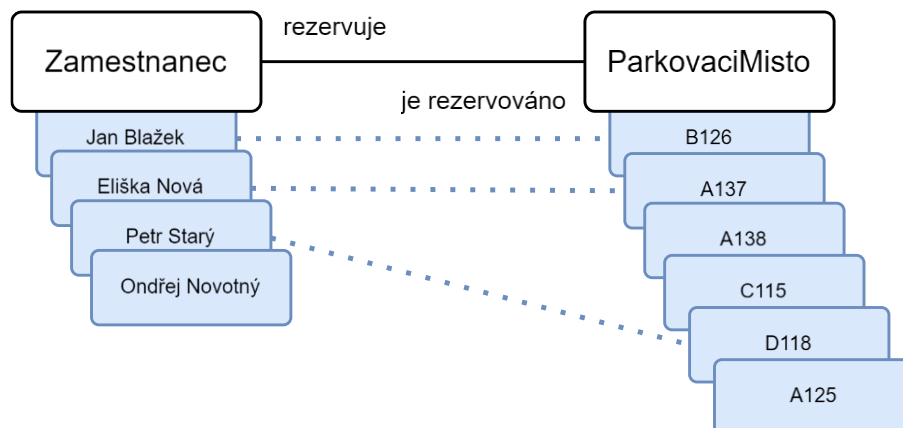
- **Může?** Pak se na jedné straně jedná o kardinalitu typu **N**.
- **Nemůže?** Pak se na jedné straně jedná o kardinalitu typu **1**.

Samozřejmě se ptáme opět v obou směrech. Výsledkem dotazování pak mohou být tři různé situace:

- Na obou stranách vztahu kardinalita typu 1 = kardinalita typu **1:1**
- Na jedné ze stran kardinalita typu 1, zatímco na druhé straně typ N = kardinalita typu **1:N**
- Na obou stranách vztahu typ N = kardinalita typu **M:N**

4.4.2 Kardinalita typu 1:1

Kardinalitu typu 1:1 použijeme v případě, kdy jedné instanci z entitního typu A přiřazujeme **maximálně jednu** instanci z entitního typu B, a naopak. Tento typ je zřejmě nejméně využívaným, avšak v reálných projektech se s ním přesto setkáme. Mějme entitní typy **Zamestnanec** a **ParkovaciMisto**.



V ukázkách do konce této kapitoly se prozatím oprostíme od grafických značek na koncích čar. Místo toho se zaměříme pouze na podstatu minima a maxima, tedy na **povinnost** členství a **kardinalitu**. Zákazník nám na schůzce sdělil, že v novém systému chtějí evidovat zaměstnance a parkovací místa. Řekl nám také, že každý zaměstnanec **nemusí** nutně mít rezervované parkovací místo (někteří auto nemají) a zároveň každé parkovací místo **nemusí** být rezervováno zaměstnancem (mají dostatek parkovacích míst). Na základě těchto informací vyhodnocujeme, že oba entitní typy mají **volitelné** (nepovinné) členství ve vztahu.

Dále jsme se dozvěděli, že každý zaměstnanec u nich ve firmě smí rezervovat **maximálně jedno** parkovací místo a zároveň jedno parkovací místo může být rezervováno **maximálně jedním** zaměstnancem na jeho jméno. Z tohoto jsme schopni odvodit, že kardinalita vztahu mezi entitními typy Zamestnanec a ParkovacíMisto je **1:1** (jedna ku jedné).

4.4.3 Kardinalita typu 1:N

Tento typ kardinality je pravděpodobně tím nejpoužívanějším. Použijeme jej v případě, kdy:

- **Jedné** instanci z entitního typu A přiřazujeme **více** instancí z entitního typu B.

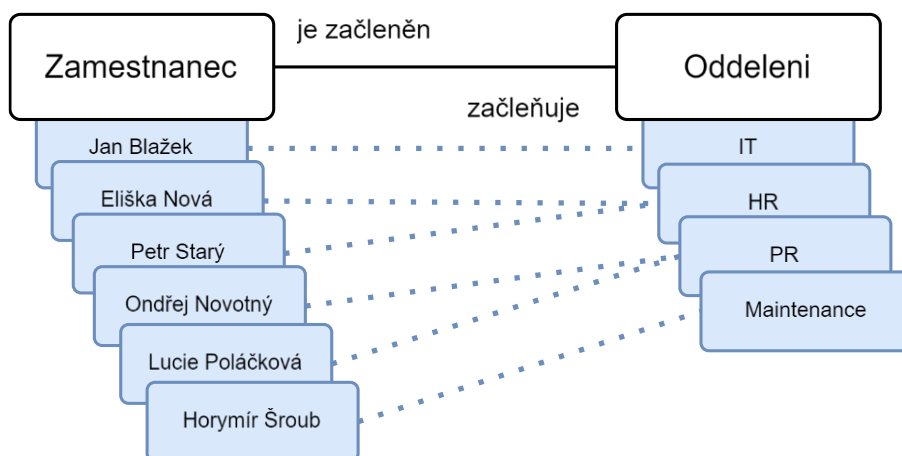
NEBO

- **Jedné** instanci z entitního typu B přiřazujeme **více** instancí z entitního typu A.



Ne však obojí, pouze jeden **nebo** druhý případ.

Pojďme se rovnou podívat na ukázkou.

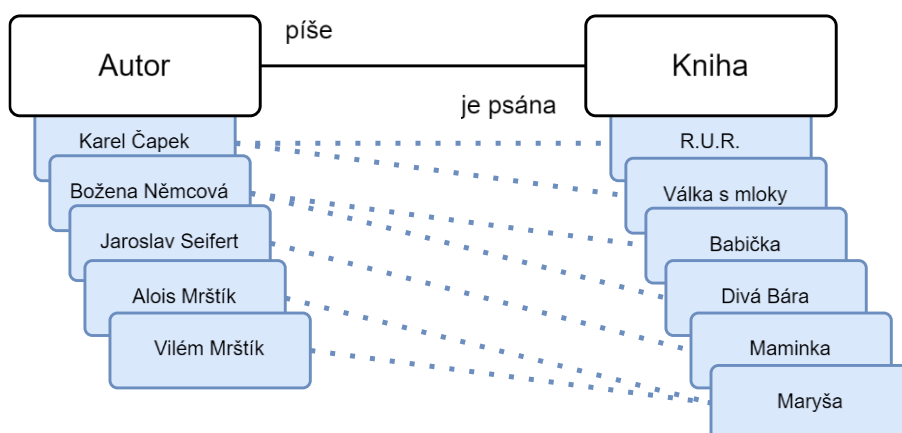


Opět jsme se zákazníkem, který potřebuje v systému tentokrát evidovat zaměstnance a oddělení. Od zákazníka se dovídáme, že v jejich firmě každý zaměstnanec **musí být členem** některého z oddělení a zároveň může dojít k situaci, kdy **oddělení nemusí** mít žádné zaměstnance (například nově vybudované oddělení). Z toho odvodíme, že entitní typ Zamestnanec má **povinné** členství ve vztahu, zatímco entitní typ Oddeleni má členství ve vztahu **volitelné**.

Dále se dovídáme, že každý zaměstnanec může být členem **maximálně jednoho** oddělení, avšak oddělení může začleňovat **více** zaměstnanců, což je u nich ve firmě naprosto běžné. Kardinalita tohoto vztahu bude tedy **1:N** (jeden k mnoha).

4.4.4 Kardinalita typu M:N

Tento typ využíváme v případě, kdy chceme umožnit každé instanci z entitního typu A přiřazení libovolného počtu instancí z entitního typu B **a zároveň i opačně**. Koukněme se na příklad s autory a knihami.



Zákazník nám sdělil, že potřebuje nový systém do knihkupectví, v němž by bylo možno evidovat knihy a autory. Podle zákazníka by měl systém fungovat tak, že autor **musí** mít alespoň jednu přiřazenou knížku, avšak knížka **nemusí** mít přiřazeného žádného autora (např. neznámý autor). Entitní typ Autor bude mít tedy **povinné** členství ve vztahu, zatímco Kniha bude mít členství ve vztahu **volitelné**.

Od zákazníka jsme se také dozvěděli, že každý autor může mít na svém kontě více knih. A dokonce i libovolná kniha může být sepsána více autory současně. Kardinalita tohoto vztahu je tedy **M:N** (mnoho k mnoha).

4.4.5 Grafický zápis kardinality

Nyní se vrhněme na symboly pro zápis kardinality. Symboly pro kardinalitu se umísťují na konec čáry, mezi entitním typem a symbolem pro povinnost členství. Takže pokud budeme mít na diagramu 2 entitní typy spojené jednou čarou, pak na obou koncích čáry uvedeme po dvou symbolech. Vždy jeden pro kardinalitu a jeden pro povinnost členství.

Kardinalita 1:1

Pro kardinalitu typu **1:1** budou na obou koncích čáry dvě čárky kolmo k čáře znázorňující vztah.



Kardinalita 1:N

Pro kardinalitu typu **1:N** bude na jednom konci čárka kolmo k čáře znázorňující vztah a na druhém **spojnice**, které se v českém IT slangu říká také „**kuří nožka**“. V anglickém IT slangu pak „**crow's foot**“, neboli vraní noha.



Kardinalita M:N

A nakonec kardinalita typu **M:N**, u níž jsou spojnice zakresleny na obou koncích.



5 Konceptuální model

Konceptuální model je prvním modelem v procesu vývoje informačního systému, který zachycuje **funkční** a **informační potřeby zákazníka**. Zpravidla jej vypracovává **analytik**. V této fázi se zabýváme tím, co je z **konceptního pohledu ideálním řešením**, ale už neřešíme skutečnou implementaci, tedy to, co je fyzicky možné (realizovatelné)... to nás zde zatím ještě tolik nezajímá.

Konceptuální model je obvykle vytvářen velmi obecně, takže v této první fázi ještě nezáleží na tom, jaký konkrétní DBMS (Microsoft, Oracle, MySQL...) bude použit.

5.1 ERM a ERD

ERM neboli **Entity-Relationship Model** je specifický přístup (metodika) pro tvorbu konceptuálního modelu. Je standardně používán mezi IT specialisty a návrháři databází pro vizualizaci a návrh struktury databáze. Slouží jako poctivý základ, který pomáhá v dalších fázích vývoje.

ERM operuje s **entitními typy** a **vztahy** mezi nimi. Graficky je reprezentován pomocí tzv. **ERD**, tedy Entity-Relationship Diagram, v němž entitní typy s jejich vztahy zakreslujeme.

Kroky k vytvoření konceptuálního modelu:

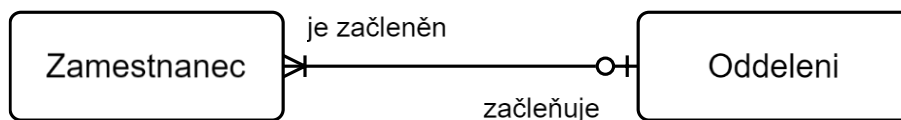


Pojďme si uvést první příklad.

Představme si, že pracujeme jako konzultant ve společnosti, která se zabývá vývojem softwaru. Na základě schůzky se zákazníkem zjistíme, že bude chtít, abychom mu naprogramovali systém na evidenci jeho zaměstnanců. Dále nám řekne, že každý zaměstnanec je evidován pod nějakým oddělením. Dozvídáme se, že každý zaměstnanec **musí** být členem nějakého oddělení, avšak každé oddělení **nemusí** mít zaměstnance. Nakonec se ještě zákazníka doptáme a získáváme informace, že každý zaměstnanec bude spadat právě pod **jedno** oddělení, a také to, že každé oddělení může začleňovat **více** zaměstnanců.

Z těchto prvotních informací můžeme sestavit ERM na základě uvedených kroků:

1. **Identifikace entit** – na základě získaných informací zakreslíme entity **Zamestnanec** a **Oddeleni**
2. **Určení povinnosti členství** – Zamestnanec má **povinné** členství ve vztahu, Oddeleni **volitelné**
3. **Určení kardinality** – kardinalita bude typu 1:N, kdy 1 oddělení může začleňovat N zaměstnanců



Rozeberme si ukázkou s entitními typy Zamestnanec a Oddeleni. Jak bylo již popsáno v minulé kapitole, symboly, které jsou umístěny vždy blíže k entitnímu typu řeší kardinalitu. Symboly vzdálenější od entitního typu vizualizují povinnost členství ve vztahu.

V ukázce vidíme, že na straně entitního typu Zamestnanec jsou zakresleny symboly „spojnice“ a „čárka“. Spojnice symbolizuje **kardinalitu typu N**, zatímco čárka symbolizuje **povinné členství** ve vztahu s entitním typem Oddeleni.

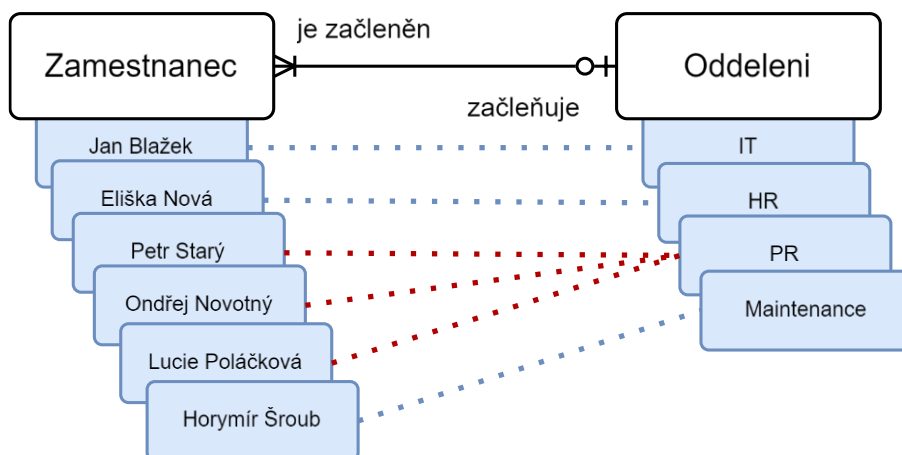
U entitního typu Oddeleni jsou zakresleny symboly „čárka“ a „kolečko“. Čárka symbolizuje **kardinalitu typu 1** zatímco kolečko symbolizuje **volitelné členství** ve vztahu s entitním typem Zamestnanec.

5.2 Symbolika kardinality

Zkusme se blíže podívat na tvar obou symbolů pro kardinalitu. Entitní typ Zamestnanec má u sebe zakreslenou **spojnici**. Oddeleni má u sebe zakreslenou **čárku**. Spojnice má tvar jakési vidličky... je to tvar, který **míří na více cílů**. Vlastně nám říká, že oddělení může mít vztah s **více zaměstnanci**. Tedy, kterékoli oddělení může mít navázán vztah s **více zaměstnanci**. Proto je spojnice zakreslena na straně entitního typu Zamestnanec.



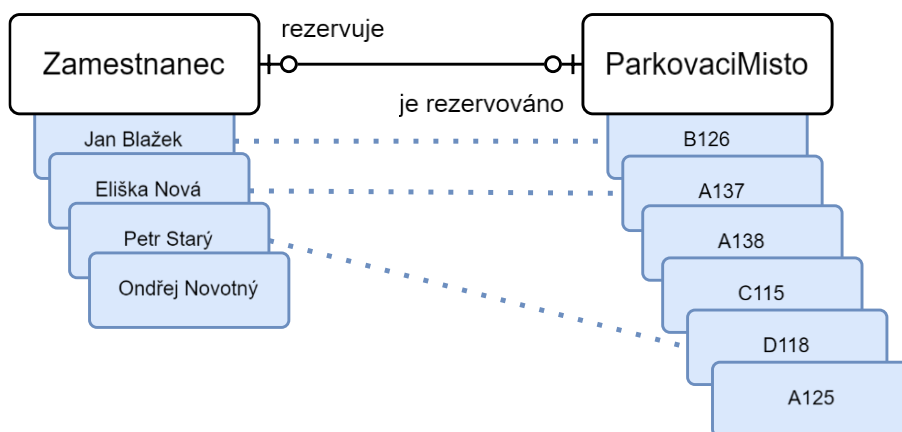
Kardinalita se zapisuje vždy na protější stranu, což znamená, že určuje, kolik zástupců druhého entitního typu může být v tomto vztahu.



Když se podíváme na spojnici a červeně vyznačené vztahy mezi třemi náhodnými zaměstnanci a oddělením PR, zjistíme, že **tvar i směr spojnice přesně kopíruje tuto realitu**.

A naopak u entitního typu oddělení je kardinalita zakreslena svislou čárkou. Ta vlastně symbolizuje, že zaměstnanec může být nanejvýš v jednom oddělení. Svislá **čárka zde symbolizuje jedničku**, což odpovídá kardinalitě typu 1.

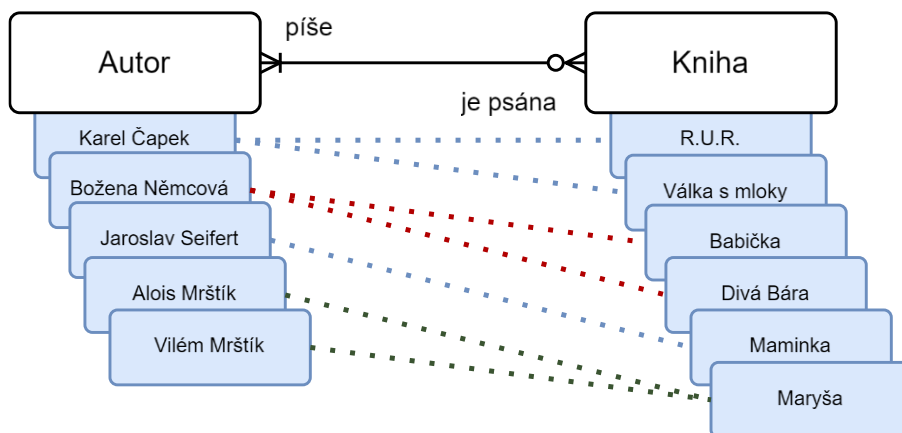
Zkusme si uvést příklad se zaměstnanci a parkovacími místy z předchozí kapitoly.



Tento diagram je doplněn o symboly kardinality i povinnosti členství. Z předchozí kapitoly již víme, že zaměstnanci nemusí mít nutně rezervovaná parkovací místa a také, že parkovací místo nemusí být nikým rezervováno. Takže členství ve vztahu by bylo pro oba entitní typy **volitelné**... tedy kolečko.

Každý zaměstnanec může mít rezervováno maximálně jedno parkovací místo. Parkovací místo může být rezervováno maximálně jedním zaměstnancem. Typ kardinality je tedy pro oba entitní typy také shodný, a to **1:1** (jedna ku jedné), tedy čárka.

A co příklad s autory a knihami?



Tento příklad jsme si rovněž vypůjčili z předchozí kapitoly, v níž jsme se od zákazníka dozvěděli, že každý autor zaevidovaný v systému **musí** mít nějakou knihu, avšak knihy **nemusí** mít nutně svého autora. Povinnost členství je tedy pouze na straně autora (čárka) zatímco knihy mají členství ve vztahu volitelné (kolečko).

A nyní kardinalita. Kterýkoli autor může napsat i **více** než jednu knihu, např. Božena Němcová má v našem systému přiřazeny 2 knihy. A dokonce i v opačném směru, kniha může být napsána **více** autory, podívejme se na Aloise a Viléma Mrštíka, ti spolu napsali Maryšu. V tomto konkrétním případě budeme hovořit o kardinalitě typu **M:N** (mnoho k mnoha).

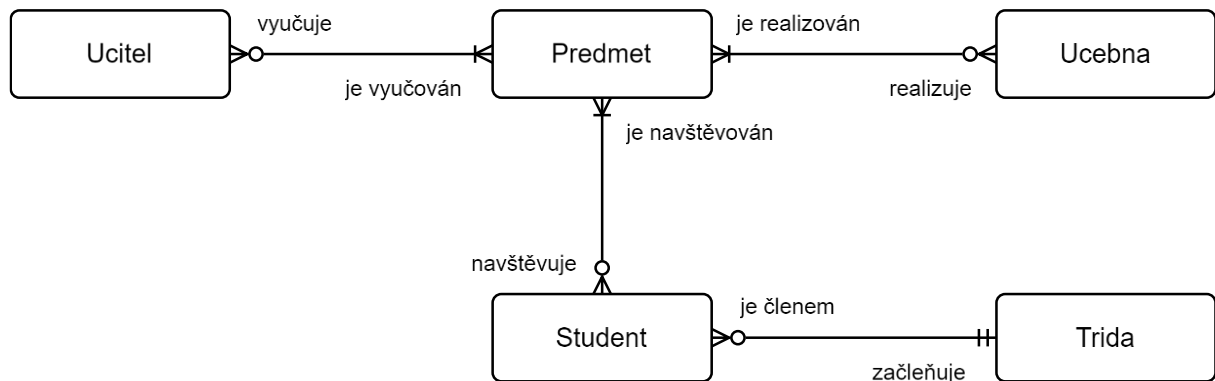
Poznámka

V tomto případě platí, že každá kniha nemusí mít svého autora. I když na obrázku není vidět žádná taková knížka bez autora, dnes tam není, za týden se může v systému objevit. Důležité pro nás jsou požadavky zákazníka. Pokud si to takto zákazník přeje, pak si je zřejmě vědom, že k situaci, kdy bude kniha bez autora může kdykoliv dojít.

Rovněž je potřeba si uvědomit, že v ERD jsou zakresleny pouze **entitní typy** a **vztahy** mezi nimi. Instance těchto entitních typů, které jsou pro účely této kapitoly symbolizované modrou barvou, se v ERD nenacházejí. Instance entitních typů si musíme během vývoje konceptuálního modelu představit ve své hlavě.

5.3 Příklad konceptuálního modelu

Následující ERD vizualizuje minimalistický školní model, v němž se nachází pět entitních typů.



5.3.1 Rozbor povinnosti členství

Učitel **nemusí** učit předmět (může být dlouhodobě nemocen, avšak stále evidován v systému).

Každý předmět **musí** být vyučován nějakým učitelem.

Každý předmět **musí** být realizován v některé z učeben (nemůžeme se učit na ulici).

Učebna **nemusí** realizovat výuku žádného předmětu (třeba je letos nevyužitá).

Každý předmět **musí** být navštěvován nějakými studenty.

Student **nemusí** navštěvovat předmět (může mít třeba přerušené studium).

Student **nemusí** být členem třídy (ze stejného důvodu).

Třída **musí** začleňovat nějaké studenty.

5.3.2 Rozbor kardinality

Učitel může vyučovat **více** předmětů.

Předmět může být vyučován **více** učiteli (Matematika je ve škole např. vyučována několika učiteli).

Předmět může být realizován ve **více** učebnách (v pondělí jsme v jedné učebně, ve středu chodíme jinam).

Učebna může realizovat **více** předmětů.

Předmět může být navštěvován **více** studenty.

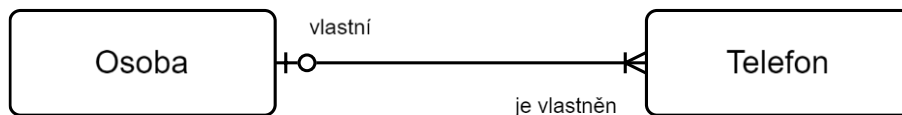
Student může navštěvovat **více** předmětů.

Student může být členem maximálně **jedné** třídy.

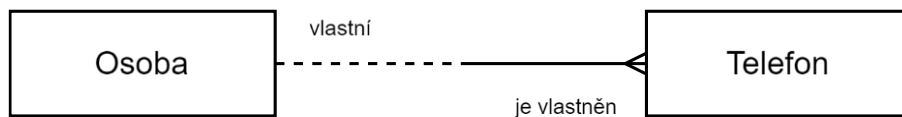
Třída může začleňovat **více** studentů.

5.4 ERD notace

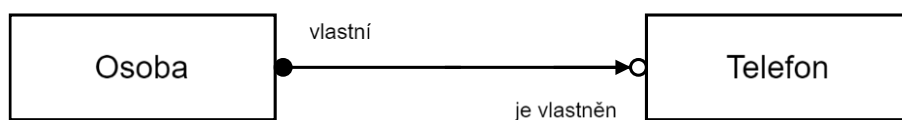
Symbols a vlastně celkový způsob, jakým zakreslujeme entitní typy a vztahy mezi nimi, nazýváme tzv. notací. Notace je systém symbolů a pravidel používaný k vizuálnímu nebo formálnímu zápisu. Notací, které je možno při tvorbě ERM použít je více. Místo obsáhlého výkladu si uvedeme jen několik vizualizací pro porovnání. Připomínám, že v rámci tohoto předmětu používáme **Information Engineering Notation**.



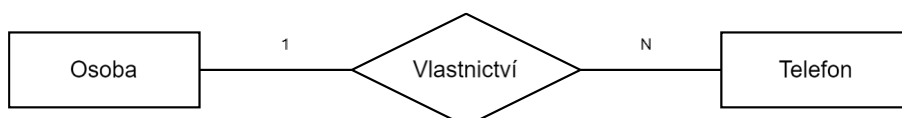
Information Engineering Notation



Barker Notation



Bachman Notation



Chen Notation

6 Klíče

V databázových systémech jsou klíče důležitým nástrojem pro organizaci a správu dat. Existuje několik typů klíčů, z nichž každý má svůj specifický účel. Některé klíče slouží přímo k identifikaci záznamů, jiné pomáhají zajistit integritu dat a propojit různé tabulky. V následujících podkapitolách si vysvětlíme jednotlivé typy klíčů, jejich vlastnosti a konkrétní využití v praxi.

6.1 Unikátní klíč

Unikátní klíč (**UID** či **unique identifier**) je takový atribut entitního typu, který zajišťuje, že hodnota tohoto atributu bude unikátní napříč všemi instancemi entitního typu. Využijeme jej všude tam, kde potřebujeme zajistit, aby se hodnoty atributu neopakovaly.

Příklad:

Typické využití unikátního klíče by mohlo být např. v tabulce pro uživatele, jejichž email bude využit zároveň k přihlášení do systému, a tudíž by neměl být využíván nikým dalším:

Jmeno	Prijmeni	Email (UID)	Registrace
Petr	Starý	stary@email.cz	2022-10-12
Eva	Modrá	modra@gmail.com	2024-05-07
Libor	Černý	cerny@email.cz	2023-12-01

Dalším příkladem by mohlo být např. telefonní číslo zákazníků, které by neměl vlastnit nikdo další v celé databázi:

Jmeno	Prijmeni	Telefon (UID)	VipClenstvi
Petr	Starý	+420606587415	ne
Eva	Modrá	+420603698524	ano
Libor	Černý	+420774325981	ano

6.2 Primární klíč

Primární klíč (**PK** či **primary key**) je zvolený unikátní klíč, který slouží k jednoznačné identifikaci řádku v tabulce.

- **Primární klíč je povinný** – musí být vždy vyplněn
- **Hodnota primárního klíče by se neměla měnit** po celou dobu existence instance

Příklad:

Mějme tabulku, která obsahuje např. dva unikátní klíče, email a rodné číslo. Oba by mohly jednoznačně identifikovat záznamy v tabulce. Takovým atributům se mimochodem říká **kandidátní klíče** (oba jsou potenciálními kandidáty na primární klíč). Z těchto kandidátních klíčů je vybrán **primární klíč** (PK) pro jednoznačnou identifikaci řádků v tabulce, zatímco zbylý kandidát se stává tzv. **sekundárním klíčem**.

Jmeno	Prijmeni	Email (UID)	RodneCislo (UID)
Petr	Starý	stary@email.cz	19910107/1628
Eva	Modrá	modra@gmail.com	19270126/6352
Libor	Černý	cerny@email.cz	19761123/2456



Pokud některý z unikátních klíčů (UID) vybereme jako primární klíč, pamatujme na to, že jeho vyplnění je vždy **povinné**, a navíc bychom měli zajistit, že se jeho hodnota nikdy **nezmění**.

6.3 Složený klíč

Složeným klíčem je myšlena kombinace **dvou nebo více atributů, které dohromady tvoří unikátní klíč (UID)**. Přestože mohou být využívány i v dnešních moderních databázových systémech, od jejich použití se často upouští kvůli následné složitější manipulaci s daty a možnému snížení výkonu systému během operací indexování a vyhledávání. Složené klíče se využívaly především v minulosti, kdy bylo potřeba šetřit paměť počítačů, která byla drahá a nebylo zcela žádoucí jen tak si do tabulek přidávat atributy, které by obsadily další paměť.

Příklad:

Koukněme se na následující ukázkou. Entitní typ s názvem **Koncert** zahrnuje 2 atributy, **DatumKonani** a **CisloSedadla**. Způsob, jakým můžeme dosáhnout jednoznačné identifikace každé instance, je v tomto případě vytvoření složeného klíče.

Kdybychom se pokusili jednotlivé instance jednoznačně identifikovat pouze pomocí atributu **DatumKonani**, neuspěli bychom, poněvadž hodnoty tohoto atributu se u entitního typu **Koncert** opakují. Stejně by tomu bylo i v případě, kdybychom se pokusili o použití pouze druhého atributu, tedy **CisloSedadla**, jako unikátního klíče. I sedadla se napříč instancemi mohou opakovat.

Když ovšem vytvoříme složený klíč, zahrnující atributy **DatumKonani** i **CisloSedadla**, tato jednoznačná identifikace každé instance bude fungovat.

Toto řešení bude funkční však jen tehdy, pokud se bude v jednom dni konat vždy nanejvýše jeden koncert. V opačném případě by tento postup selhal a došlo by k opakujícím se hodnotám a my bychom nedokázali jednoznačně rozlišit dva a více identických záznamů (instancí).

DatumKonani	CisloSedadla
20.10.2024	10
20.10.2024	28
20.10.2024	14
27.10.2024	14

6.4 Umělý klíč

Dnes je prakticky běžnou praxí, že se pro jedinečnou identifikaci instancí využívají tzv. **umělé klíče**. Umělý klíč je atribut, který není přirozenou součástí dané entity, ale **slouží k jednoznačné identifikaci záznamu**. Tento speciální klíč se vytvoří přidáním nového atributu, který je často označován jako **id**, **Id** či **ID**. Hodnoty umělého klíče mohou být automaticky generovány databázovým systémem, což znamená, že databázový administrátor ani programátor se nemusí starat o jejich ruční zadávání.

Příklad:

V tabulce se zaměstnanci může dojít k situaci, kdy se některé jméno, např. Petr Novák, objeví vícekrát. Pokud bychom chtěli upravit nebo vymazat záznam s tímto jménem, nebylo by zřejmé, o který záznam se jedná. Proto je důležité, aby tabulka obsahovala **primární klíč**. Jak jsme si již zmínili v předchozím textu, primární klíč bývá obvykle vytvořen z existujícího unikátního klíče. Co však dělat, pokud tabulka (entitní typ) neobsahuje žádný vhodný atribut, který by mohl sloužit jako primární klíč? V takovém případě je nutné do entitního typu přidat nový atribut, tzv. **umělý klíč**. Tento atribut, často označovaný jako **Id**, se postará o jednoznačnou identifikaci záznamů v tabulce.

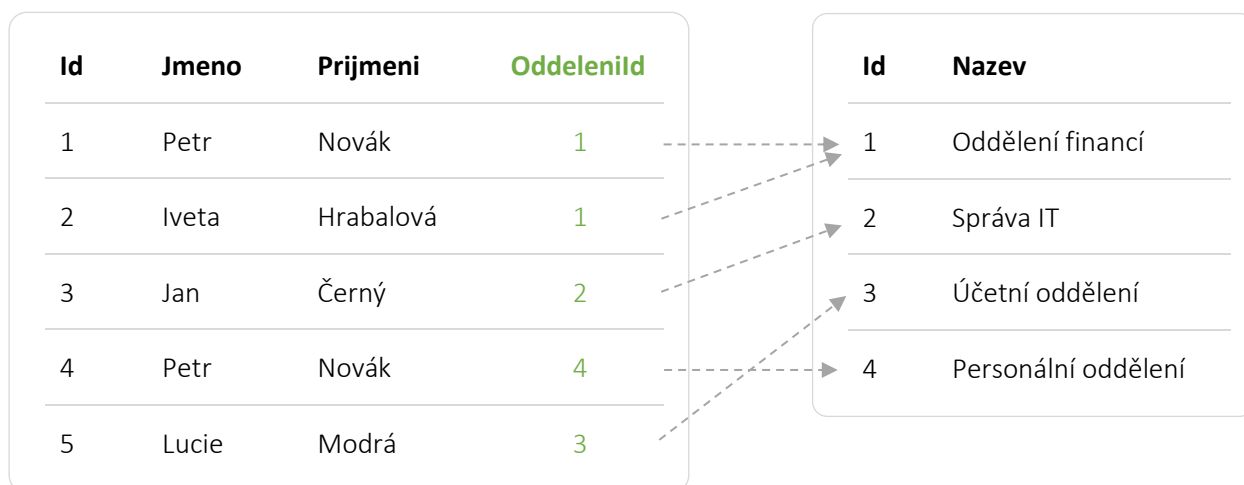
Id	Jmeno	Prijmeni	Funkce	Kancelar
1	Petr	Novák	Finance	12
2	Iveta	Hrabalová	Finance	12
3	Jan	Černý	IT	11
4	Petr	Novák	Personální	8

6.5 Cizí klíč

Cizí klíč (foreign key či FK) je odkaz na primární klíč v jiné tabulce. Je to vlastně způsob, jak propojit dvě tabulky v databázi, aby mezi nimi vznikl vztah. Cizí klíč je v tabulce, která potřebuje odkazovat na jinou tabulku. Tento klíč obsahuje hodnoty, které odpovídají hodnotám v primárním klíči jiné tabulky.

Příklad:

Mějme tabulku se zaměstnanci a tabulku pro jednotlivá oddělení. Každý zaměstnanec spadá pod některé oddělení. Takže záznam zaměstnance bude vždy obsahovat také odkaz na oddělení, pod které daný zaměstnanec spadá:



Z ukázky je patrné, že každá instance zaměstnance má vztah s nějakým oddělením. Iveta Hrabalová je například umístěna pod oddělení financí, Jan Černý pod Správu IT atd. Takto jsou v relačních databázích realizovány vztahy mezi instancemi.

7 Logický model

Logický model je dalším modelem v procesu vývoje informačního systému, který bývá připravován **návrhářem databází** (nebo systémovým analytikem). Tento model navazuje na model konceptuální a rozšiřuje ho o některé další detaily, které v konceptuálním modelu nejsou. Stejně jako konceptuální model, i logický model bývá vizualizován pomocí diagramu typu **ERD** (Entity-Relationship Diagram).

Logický model se soustředí na strukturu dat, vztahy mezi nimi a jejich vlastnosti. Obsahuje následující komponenty:

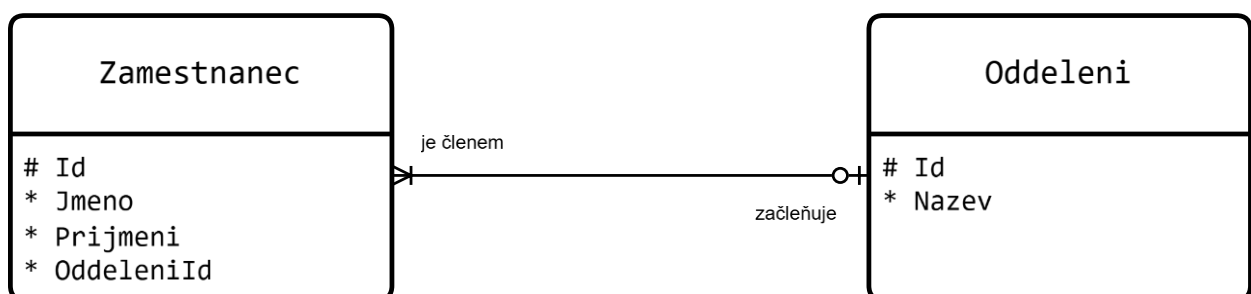
- **Entitní typy**
- **Vztahy** (včetně povinnosti členství a kardinality)
- **Atributy**
- **Klíče** (primární a cizí klíče, které slouží k identifikaci a propojení entit v databázi)

7.1 Dohodnutá konvence

Dohodněme se nyní na způsobu, jakým budeme vytvářet diagram ERD v rámci logického modelu. V níže umístěné ukázce je pomocí ERD znázorněn logický model, který obsahuje všechny potřebné komponenty. Každý entitní typ je symbolizován obdélníkem, který je rozdělen na dvě části: prostor pro **název** entitního typu a prostor pro jeho **atributy**.

V rámci logického modelu mohou být atributy entitních typů znázorněny různými symboly, které zpřehledňují zápis:

- **Symbol hash (#)** – používáme pro označení primárního klíče
- **Symbol hvězdička (*)** – označuje povinné atributy
- **Symbol prázdné kolečko (o)** – označuje nepovinné atributy



Z logického modelu lze vyčíst, že entitní typ **Zamestnanec** obsahuje povinné atributy *Id*, *Jmeno*, *Prijmeni* a *OddeleniId*, představující cizí klíč, který odkazuje na primární klíč tabulky *Oddeleni*. Entitní typ **Oddeleni** obsahuje povinné atributy *Id* a *Nazev*.

Dále vidíme to, co je již typické pro předchozí konceptuální model, tedy vztah mezi oběma entitními typy, povinnost členství ve vztahu a kardinalitu vztahu. Zaměstnanec je členem oddělení. Oddělení začleňuje zaměstnance. Každý zaměstnanec musí být členem některého oddělení, a to právě jednoho. Oddělení může existovat i bez zaměstnanců, ale může jich začleňovat klidně i více.



Název cizího klíče budeme vždy skládat z názvu tabulky, na níž cizí klíč odkazuje a dále z názvu primárního klíče tabulky, na níž je odkazováno. Například tak jako ve výše uvedené ukázce s cizím klíčem **OddeleniId**.

Dalším příkladem může být zaměstnanec a telefon, přičemž každý zaměstnanec může mít více telefonních čísel, ale nemusí mít žádné, zatímco každé telefonní číslo musí být někým vlastněno a může být vlastněno maximálně jedním zaměstnancem. Zaměstnanec má povinné atributy *Id*, *Jmeno*, *Prijmeni* a nepovinný atribut *Poznamka*. Telefon má povinné atributy *Id*, *Predvolba*, *Cislo* a nakonec *ZamestnanecId*, což je cizí klíč odkazující na tabulku *Zamestnanec* a její primární klíč s názvem *Id*.



7.2 Dekompozice vztahu M:N

V rámci konceptuálního a logického modelování pracujeme běžně se vztahy M:N. Potíží je však v tom, že v reálném relačním databázovém systému není tento typ vztahu implementovatelný. Zkrátka není způsob, jak bychom realizovali vztah typu M:N přímo mezi dvěma tabulkami.

Řešením tohoto problému je tzv. **dekompozice**, což je proces, který složitý vztah s kardinalitou M:N převádí na dva jednodušší vztahy s kardinalitou 1:N. To znamená, že místo dvou tabulek s přímým vztahem (M:N), vytvoříme tři tabulky, kde jedna tabulka bude sloužit jako propojující tabulka s cizími klíči na obě původní tabulky.

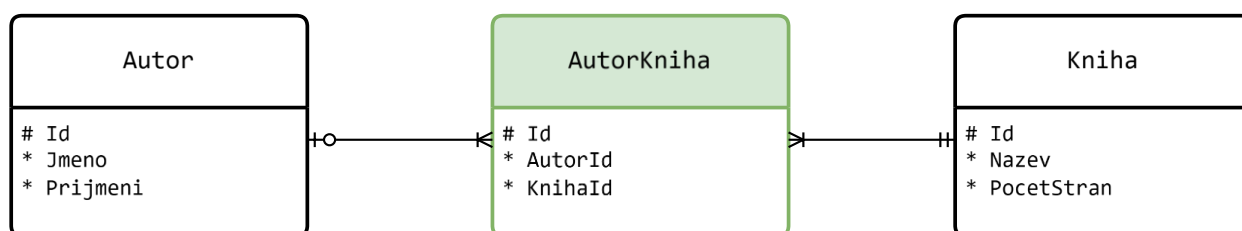
Uvedme si příklad na autorovi a knize. Autor může napsat více knih a zároveň platí, že i každá kniha může být napsána více autory. V konceptuálním modelování je to hračka. Avšak jakmile se dostáváme k logickému modelu, v němž figuruje i cizí klíč, vyvstává otázka, kam ten cizí klíč přiřadit.

Pokud bychom cizí klíč vytvořili na straně autora, pak by každý autor mohl napsat maximálně jednu knížku. Pokud bychom to udělali opačně, tak by zase každá knížka mohla mít maximálně jednoho autora. Ani jedna varianta neřeší naši situaci. Proto je potřeba sáhnout po dekompozici vztahu M:N.



Jak bylo již řečeno, dekompozice převádí složitější vztah s kardinalitou M:N na dva jednodušší vztahy s kardinalitou 1:N. K tomu je využita tzv. **vazební tabulka** (nebo také **intersection entity**), která má za úkol spojovat instance obou původních entitních typů. Tato vazební tabulka obsahuje kromě vlastního primárního klíče také cizí klíče, které odkazují na spojované tabulky. Pokud má být zabezpečen vztah mezi tabulkou **Autor** a **Kniha**, pak vazební tabulka bude obsahovat dva cizí klíče, **AutorId** a **KnihaId**.

Co se týká pojmenování vazební tabulky, to může být tvořeno například sloučením názvů obou spojovaných tabulek, případně jiným způsobem, který bude mít vypovídající hodnotu.



Pro každý vztah mezi autorem a knihou vznikne ve vazební tabulce jeden záznam. Takže pokud autor napsal 3 nějaké knížky, pak ve vazební tabulce přibydou 3 záznamy, přičemž každý takový záznam bude obsahovat **Id autora** a **Id knížky**, kterou napsal.

7.2.1 Povinnost členství v dekomponovaném modelu

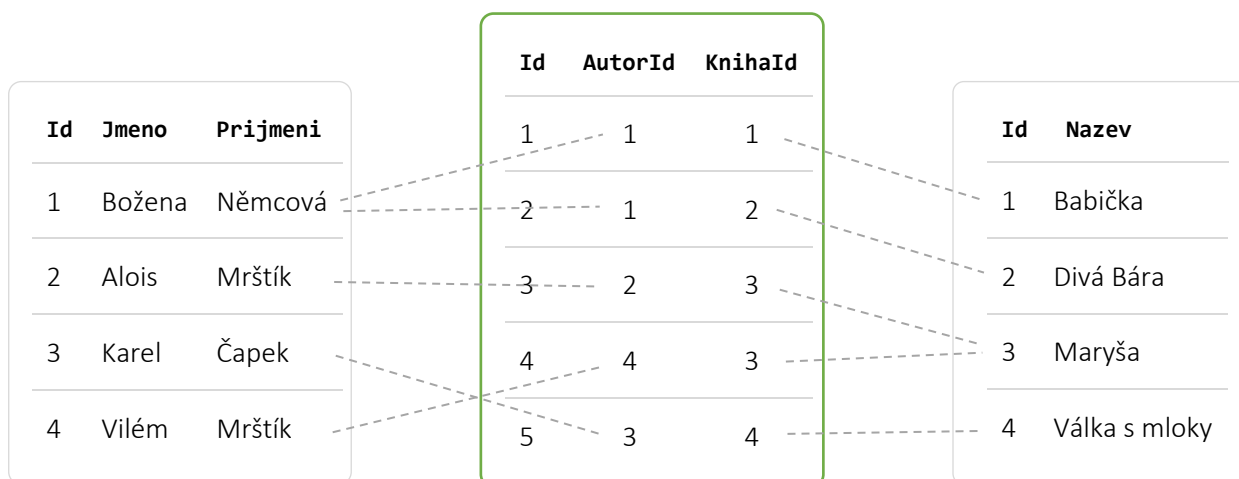
Všimněme si, že **vazební tabulka** z poslední ukázky má na obou svých stranách označeno **povinné členství** ve vztahu. Pokud by hodnoty cizích klíčů ve vazební tabulce nebyly povinné, záznam ve vazební tabulce by ztratil smysl. Jen si to představme... autor Božena Němcová ve vztahu bez přiřazené knihy, nebo kniha Babička bez přiřazeného autora. Aby vazební tabulka efektivně plnila svou funkci a správně propojovala záznamy ze dvou tabulek, je nutné, aby hodnoty cizích klíčů ve vazební tabulce byly vždy povinné.

Při pohledu na původní tabulky (Autor a Kniha) vidíme, že jejich kardinalita byla upravena tak, aby na straně tabulek byla symbolizována „jedničkou“ – tedy svislou čárkou. Povinnost členství ve vztahu je však nyní zajištěna vazební tabulkou. To znamená, že původní tabulky již přímo nerozhodují o povinnosti svého členství ve vztahu.

Přesto se doporučuje **symboly povinného členství u původních tabulek zachovat**. Tyto symboly poskytují důležitou informaci pro ostatní členy týmu, zejména pro programátory. Například při ukládání nové knihy do systému by měl být uživatel programem veden k výběru autora knihy před uložením záznamu.

7.2.2 Tabulkové zobrazení

Pojďme si ještě ukázat, jak je logický model realizován v relační databázi. K tomu nám poslouží tabulkové zobrazení. Dekomponovaný model obsahuje kromě dvou původních tabulek také vazební tabulku s cizími klíči odkazujícími na primární klíče obou tabulek. Každý vztah mezi autorem a knihou má ve vazební tabulce svůj vlastní záznam.



8 Normalizační proces

Normalizace je proces organizování atributů a tabulek relační databáze, který minimalizuje redundanci (nadbytečnost) dat. Proces normalizace pomáhá zvládat anomálie, ke kterým by mohlo docházet během vkládání, editace nebo mazání dat. A také zajišťuje lepší výkon databáze.

Normalizační proces je realizován prostřednictvím tzv. **normálních forem**, což můžeme chápat jako soubor určitých pravidel. Přestože existuje více normálních forem, nejčastěji se dodržují první 3 z nich, což dostatečně zajišťuje správný návrh databáze.

8.1 První normální forma (1NF)

Tabulka splňuje první normální formu (1NF), pokud platí následující:

- **Všechny atributy (sloupce) jsou atomické** (dále nedělitelné).
- **V tabulce nejsou duplicitní záznamy** (řádky).
- **Tabulka má primární klíč (PK)**, který zajišťuje jedinečnost každého řádku.

8.1.1 Úpravy v rámci jedné tabulky

Někdy stačí upravit strukturu jedné tabulky, aby splňovala 1NF. V jiných případech je nutné tabulku rozdělit na více menších tabulek. Vše závisí na původním návrhu tabulky. U jednodušších tabulek často postačí drobné úpravy, zatímco složitější tabulky mohou vyžadovat větší zásahy.

Příklad 1 – Tabulka Zaměstnanec

Pojďme se podívat na příklad tabulky `Zamestnanec`, která nesplňuje 1NF. V následujícím příkladu má tabulka tyto problémy:

1. Sloupec `Jmeno` obsahuje více dat (křestní jméno a příjmení).
2. Sloupec `Kontakt` obsahuje směs různých typů dat (telefonní číslo a email).
3. Tabulka nemá primární klíč (PK), který by zajistil jedinečnost každého řádku.

Kdybychom potřebovali takto uložená data z databáze načíst do programu a následně vypsát všechna křestní jména zaměstnanců, pravděpodobně by se jednalo o nadlidský úkol. Program totiž neví, co je jméno a co je příjmení. A programátor, ten vůbec předem netuší, jaká jména a příjmení na něj z databáze nakonec vyskočí. A co teprve s kontakty? Pokud by měl program za úkol vypsát telefonní čísla nebo jen emaily z takovéto datové divočiny, byla by to fuška a program by byl navíc velmi neefektivní.

Jmeno	Kontakt
Petr Novák	+420602657521, novak@email.cz
Iveta Hrabalová	+420606123321, hrabalova@gmail.com
Jan Černý	+420607258937, cerny@seznam.cz
Petr Novák	+420774033027, novak@gmail.com

Řešení:

- Rozdělíme sloupce na atomické hodnoty:
 - Sloupec **Jmeno** rozdělíme na Křestní jméno a Příjmení.
 - Sloupec **Kontakt** rozdělíme na Telefon a Email.
- Přidáme nový sloupec **Id**, který bude sloužit jako primární klíč.

Zamestnanec				
Id	Jmeno	Prijmeni	Telefon	Email
1	Petr	Novák	+420602657521	novak@email.cz
2	Iveta	Hrabalová	+420606123321	hrabalova@gmail.com
3	Jan	Černý	+420607258937	cerny@seznam.cz
4	Petr	Novák	+420774033027	novak@gmail.com

Tato podoba tabulky již splňuje 1NF. Každý řádek v tabulce je jednoznačně identifikován primárním klíčem. Každý atribut nyní obsahuje pouze jeden typ dat a jednu hodnotu.

Nyní již bude snadné vypsát křestní jména zaměstnanců, oddělit je od příjmení, filtrovat nebo seřazovat záznamy podle emailu či telefonního čísla.

Příklad 2 – Tabulka Notebook

Podívejme se na další příklad, tabulku Notebook, která také nesplňuje 1NF. Tabulka má tyto problémy:

- Nemá primární klíč (PK).
- Sloupec **Detaily** obsahuje více typů dat najednou, což porušuje atomičnost.

Nazev	Detaily
NB-005	HP, 15 palců, 8 GB RAM
NB-017	Samsung, 13.3 palců, 16 GB RAM
NB-003	Apple, 14 palců, 16 GB RAM
NB-007	Dell, 15 palců, 16 GB RAM

Řešení:

1. Přidáme sloupec `Id` jako primární klíč.
2. Rozdělíme sloupec `Detaily` na více sloupců, například:
 - a. Název,
 - b. Velikost displeje (v palcích),
 - c. Operační paměť (v GB).

Notebook				
Id	Nazev	Vyrobce	VelikostDispleje	OperacniPamet
1	NB-005	HP	15	8
2	NB-017	Samsung	13.3	16
3	NB-003	Apple	14	16
4	NB-007	Dell	15	16

V relačních databázových systémech je běžné, že každý sloupec tabulky můžeme doplnit **komentářem**, který pomůže členům vývojového týmu. Tento komentář může např. obsahovat informaci o tom, v jakých jednotkách je myšlena velikost displeje (např. v palcích) nebo upřesnění, co znamená pojem `OperacniPamet` (např. kapacita operační paměti v GB). Díky tomu není zcela nutné vytvářet komplikované názvy atributů (sloupců) jako je třeba ***VelikostDisplejeVPalcich*** nebo ***KapacitaOperacniPametiVGB***. S takovým kódem by se nám později velmi špatně pracovalo.

8.1.2 Úpravy v rámci více tabulek

V některých případech je nutné tabulku rozdělit na více propojených tabulek, aby splňovala 1NF. To může být nutné, pokud:

- Data se zbytečně opakují (například vícekrát zapisujeme stejné informace).
- Existují nesouvisející atributy, které by měly být odděleny.

Příklad 1 – Zaměstnanci a oddělení

Tabulka, která nesplňuje 1NF, má následující problémy:

1. **Sloupec `Jmeno` obsahuje více dat** (křestní jméno a příjmení).
2. **Sloupec `Oddeleni` obsahuje redundantní data**, protože hodnoty se opakují na různých řádcích.
3. **Chybí primární klíč (PK)**, který by zajistil jedinečnost každého záznamu.

Jmeno	Oddeleni
Petr Novák	Finance
Iveta Hrabalová	Finance
Jan Černý	IT
Petr Novák	Finance

Řešení:

1. **Přidání primárního klíče** – doplníme sloupec `Id`, který jednoznačně identifikuje každý záznam.
2. **Rozdělení sloupce `Jmeno`** na samostatné sloupce `Jmeno` a `Prijmeni`.
3. **Vytvoření nové tabulky `Oddeleni`** – přesuneme informace o odděleních do samostatné tabulky.

Redundance dat v tabulce (opakování stejných hodnot) je v databázových systémech nežádoucí. Přesunem informací o odděleních do samostatné tabulky zajistíme, že název každého oddělení bude uložen pouze jednou. Vytvořením vazby mezi tabulkou zaměstnanců a oddělení pomocí primárního a cizího klíče umožníme efektivní správu dat. Například při přejmenování některého z oddělení stačí provést změnu pouze v konkrétním řádku tabulky `Oddeleni`.

Zamestnanec

Id	Jmeno	Prijmeni	OddeleniId
1	Petr	Novák	1
2	Iveta	Hrabalová	1
3	Jan	Černý	2
4	Petr	Novák	1

Oddeleni

Id	Nazev
1	Finance
2	IT
3	Údržba

Příklad 2 – Zaměstnanci a kontakty

V tomto příkladu má tabulka následující problémy:

1. **Sloupec** `Jmeno` **obsahuje více dat** (křestní jméno a příjmení).
2. **Sloupec** `Kontakt` **obsahuje více hodnot**, což porušuje pravidlo atomičnosti.
3. **Tabulka nemá primární klíč**, takže nelze jednoznačně identifikovat řádky.

Jmeno	Kontakt
Petr Novák	+420602657521, +420607365412
Iveta Hrabalová	
Jan Černý	+420607258937
Petr Novák	+420774033027, +420733967631, +420755663124

V následujících ukázkách si předvedeme řešení, která sice mohou vypadat jako splnění 1NF, avšak tato řešení obsahují anomálie v podobě nesmyslně opakujících se dat.

Nekorektní řešení 1 – opakující se sloupce

První nekorektní řešení sice správně doplňuje do tabulky sloupec `Id`, který bude plnit roli primárního klíče, a také rozděluje atributy pro telefonní čísla do jednotlivých sloupců. Tento návrh je však problematický, protože:

- **Není flexibilní** – Pokud má zaměstnanec více telefonů než kolik sloupců tabulka umožňuje, je nutné upravovat strukturu tabulky.
- **Plýtvá pamětí** – Většina zaměstnanců má pravděpodobně jen jeden telefon, což vede k nevyužití sloupců.

Id	Jmeno	Prijmeni	Telefon1	Telefon2	Telefon3
1	Petr	Novák	+420602657521	+420607365412	
2	Iveta	Hrabalová			
3	Jan	Černý	+420607258937		
4	Petr	Novák	+420774033027	+420733967631	+420755663124

Nekorektní řešení 2 – opakující se řádky

Tento přístup redukuje počet sloupců, ale duplikuje řádky – pro každý telefonní kontakt zaměstnance je vytvořen nový řádek. I když tento přístup umožňuje ukládat libovolný počet telefonů, přináší jiné problémy. V tabulce se zbytečně **duplikují hodnoty zaměstnanců**, a dokonce v případě stejně jmenujících se zaměstnanců nevíme, které kontakty jsou od prvního Petra Nováka a které patří druhému Petru Novákovi.

Id	Jmeno	Prijmeni	Telefon
1	Petr	Novák	+420602657521
2	Petr	Novák	+420607365412
3	Iveta	Hrabalová	
4	Jan	Černý	+420607258937
5	Petr	Novák	+420774033027
6	Petr	Novák	+420733967631
7	Petr	Novák	+420755663124

Správné řešení – rozdělení do dvou tabulek

Optimální návrh spočívá v rozdělení tabulky na dvě:

1. **Tabulka Zamestnanci** – Obsahuje jedinečné záznamy zaměstnanců.
2. **Tabulka Kontakty** – Ukládá informace o telefonních číslech a je propojena se zaměstnanci pomocí cizího klíče.

Tento návrh umožňuje:

- **Ukládat libovolný počet kontaktů** pro zaměstnance bez nutnosti měnit strukturu tabulek.
- **Minimalizovat redundanci dat** – Každý zaměstnanec i každý kontakt je uložen pouze jednou.
- **Zlepšit efektivitu** – Pokud zaměstnanec nemá žádný telefon, v tabulce *Kontakty* pro něj neexistuje záznam.

Zamestnanec

Id	Jmeno	Prijmeni
1	Petr	Novák
2	Iveta	Hrabalová
3	Jan	Černý
4	Petr	Novák

Telefon

Id	Cislo	ZamestnanecId
1	+420602657521	1
2	+420607365412	1
3	+420607258937	3
4	+420607258937	4
5	+420733967631	4
6	+420755663124	4

8.2 Druhá normální forma (2NF)

Tabulka splňuje druhou normální formu (2NF), pokud platí následující:

- Je splněna první normální forma (1NF).
- Každý atribut (který není součástí primárního klíče), je plně závislý na celém primárním klíči.



Jednotlivé **normální formy na sebe navazují**. To znamená, že pro splnění druhé normální formy (2NF) je potřeba splnit první normální formu (1NF). Podobně splnění třetí normální formy (3NF) vyžaduje, aby již byla splněna druhá normální forma (2NF), a tak dále.

Příklad 1 – Zaměstnanec a Projekt

Tabulka níže ilustruje příklad evidence zaměstnanců, kteří se podílejí na různých projektech. Primární klíč je zde složený ze sloupců `IdZamestnance` a `IdProjektu`.

V tomto příkladu má tabulka následující problémy:

1. Sloupec `JmenoZamestnance` je funkčně závislý na sloupci `IdZamestnance`
2. Sloupec `HodinovaSazba` je také funkčně závislá na sloupci `IdZamestnance`
3. Sloupec `NazevProjektu` je funkčně závislý na sloupci `IdProjektu`

Když je sloupec funkčně závislý na jiném sloupci, můžeme jinými slovy říct, že se z něj dá odvodit. Takže pokud je např. sloupec `JmenoZamestnance` funkčně závislé na sloupci `IdZamestnance`, znamená to, že jméno lze na základě `IdZamestnance` odvodit (Pokud je `IdZamestnance` 1, pak jméno bude vždy Novák).

<u>IdZamestnance</u>	<u>IdProjektu</u>	JmenoZamestnance	NazevProjektu	HodinovaSazba
1	101	Novák	Phobos	450
2	102	Dvořák	Deimos	600
1	102	Novák	Deimos	450
3	103	Starý	Artemis	700

Problém u splnění druhé normální formy (2NF) je v tom, že všechny atributy, které nejsou součástí složeného primárního klíče musí být odvoditelné pouze na základě **celého primárního klíče**, nejen z jeho některé části.

Aby tabulka splňovala 2NF, rozdělíme ji na menší tabulky tak, aby každý „neklíčový“ atribut byl plně závislý na **celém** primárním klíči.

Zamestnanec		
Id	Jmeno	HodinovaSazba
1	Novák	450
2	Dvořák	600
3	Starý	700

Projekt	
Id	Nazev
101	Phobos
102	Deimos
103	Artemis

ZamestnanecProjekt		
Id	ZamestnanecId	ProjektId
1	1	101
2	1	102
3	2	102
4	3	103

Tabulku jsme rozdělili na dvě tabulky, které obsahují související data a zároveň jsme se zbavili funkčních závislostí. Když se podíváme pozorně, zjistíme, že tabulky již neobsahují ani složený primární klíč, takže 2NF byla tímto vyřešena. Protože rozdělením tabulky na dvě menší došlo ke vztahu s kardinalitou M:N, bylo nutné přidat ještě jednu, třetí tabulku pro realizaci vazeb mezi tabulkami pro zaměstnance a projekt.

Příklad 2 – Student a Předmět

Následující příklad řeší evidenci studentů a předmětů. Složený primární klíč je zde tvořen sloupci `IdStudenta` a `IdPredmetu`.

<u>IdStudenta</u>	<u>IdPredmetu</u>	JmenoStudenta	NazevPredmetu	JmenoUcitele
1	1	Petr Snaživý	Matematika	Ivan Malý
1	2	Petr Snaživý	Fyzika	Josef Velký
2	1	Libor Dvořák	Matematika	Ivan Malý

Důvody, proč tato tabulka nesplňuje druhou normální formu (2NF) jsou následující:

1. **Tabulka nesplňuje první normální formu (1NF)**, protože `JmenoUcitele` a `JmenoStudenta` obsahuje jméno i příjmení, porušuje tedy pravidlo atomičnosti atributů.
2. **Jména studentů lze odvodit pouze z části složeného primárního klíče (`IdStudenta`)**, jméno je tedy tzv. funkčně závislé na sloupci `IdStudenta`.
3. **Hodnoty ze sloupců `NazevPredmetu` a `JmenoUcitele` lze odvodit pouze z části složeného primárního klíče (`IdPredmetu`)**, jsou na něm funkčně závislé.

Řešením bude opět rozdělení tabulky na dvě menší tabulky, které obsahují související data. A protože rozdělením i v tomto případě vznikne vztah s kardinalitou M:N, je potřeba vytvořit také vazební tabulku pro realizaci vztahů.

Student		
Id	Jmeno	Prijmeni
1	Petr	Snaživý
2	Libor	Dvořák

Predmet			
Id	Nazev	JmenoUcitele	PrijmeniUcitele
1	Matematika	Ivan	Malý
2	Fyzika	Josef	Velký

StudentPredmet		
Id	StudentId	PredmetId
1	1	1
2	1	2
3	2	1

V tomto případě by se dalo ještě něco vylepšit. Tabulka s předmětem by mohla být rozdělena na předmět a učitele. Pokud by nastala situace, kdy by jeden předmět mohlo učit více učitelů, pak by nám aktuální řešení přestalo stačit.

8.3 Třetí normální forma (3NF)

Tabulka splňuje třetí normální formu (3NF), pokud platí následující:

- Je splněna druhá normální forma (2NF).
- Každý neklíčový atribut musí být přímo závislý pouze na primárním klíči.
- Funkční závislosti mezi neklíčovými atributy nejsou povoleny.

Příklad 1 – Zaměstnanec

Tabulka níže ilustruje příklad evidence zaměstnanců, přičemž každý zaměstnanec má nějakou funkci a základní mzdu.

V tomto příkladu má tabulka následující problém:

- Mezi sloupci `Funkce` a `ZakladniMzda` je funkční závislost

Zamestnanec				
Id	Jmeno	Prijmeni	Funkce	ZakladniMzda
1	Jan	Starý	Asistent	20000
2	Karel	Zelený	Specialista	30000
3	František	Pokorný	Specialista	30000
4	Eva	Mladá	Manažer	50000

Jinými slovy, na základě hodnoty ve sloupci `Funkce` jsme schopni odvodit hodnotu sloupce `ZakladniMzda` a naopak. Každý asistent bude mít vždy základní mzdu 20 000 Kč. Každý Specialista bude mít základní mzdu 30 000 Kč atd.

Je jasné, že neplechu nám zde dělají neklíčové atributy (sloupce) `Funkce` a `ZakladniMzda`. Řešením bude, že půjdou „z kola ven“. Vytvoříme tedy další tabulku, např. s názvem `Funkce`, a v ní budeme evidovat funkci a základní mzdu pro každou tuto funkci. V původní tabulce se zaměstnanci vytvoříme sloupec s cizím klíčem, kterým obě tabulky propojíme s kardinalitou 1:N, tedy každý zaměstnanec může mít pouze jednu funkci, ale stejná funkce může být využita u více zaměstnanců.

Zamestnanec

Id	Jmeno	Prijmeni	FunkceId
1	Jan	Starý	1
2	Karel	Zelený	2
3	František	Pokorný	2
4	Eva	Mladá	3

Funkce

Id	Nazev	ZakladniMzda
1	Asistent	20000
2	Specialista	30000
3	Manažer	50000

Toto řešení omezilo duplicity v tabulce se zaměstnanci, nicméně, když se nyní podíváme na nově vytvořenou tabulku pro funkce, zjistíme, že problém stále přetrvává, jen jsme ho přesunuli jinam. V tabulce s funkcemi stále existují dva neklíčové atributy, které jsou na sobě závislé. Stále totiž platí, že z názvu funkce jsme schopni odvodit její základní mzdu. Existují možnosti, jak problém vyřešit.

8.3.1 Změna primárního klíče

Pokud bychom odebrali sloupec `Id` a jako primární klíč označili sloupec `Nazev`, pak bychom splnili třetí normální formu (3NF). Toto řešení je jednoduché, nicméně nese s sebou i určitá **rizika**, poněvadž zákazník by se mohl v budoucnu rozhodnout změnit některé názvy funkcí, čímž by došlo ke **změně hodnot ve sloupci primárního klíče**, což není dobrou praxí a nemělo by k tomu docházet.

Funkce

Nazev (PK)	ZakladniMzda
Asistent	20000
Specialista	30000
Manažer	50000

8.3.2 Další dělení tabulek

Druhým řešením bychom dodržovali třetí normální formu (3NF) skutečně do důsledku a výsledkem by byly tři propojené tabulky: **Zamestnanec**, **Funkce** a **ZakladniMzda**. Toto řešení je sice funkční, nicméně, pokud bychom v praxi postupovali tímto stylem u všech tabulek, přidělali bychom si hodně práce.

Zamestnanec

Id	Jmeno	Prijmeni	FunkceId
1	Jan	Starý	1
2	Karel	Zelený	2
3	František	Pokorný	2
4	Eva	Mladá	3

Funkce

Id	Nazev	ZakladniMzdaId
1	Asistent	1
2	Specialista	2
3	Manažer	3

ZakladniMzda

Id	Hodnota
1	20000
2	30000
3	50000

8.3.3 Praxe ukáže...

Obě řešení, která jsme probrali, splňují požadavky třetí normální formy (3NF). Nicméně je důležité vzít v úvahu, jak to vypadá v praxi. Každá tabulka a každé řešení je posuzováno individuálně, a může se stát, že některá normální forma je splněna pouze částečně. V reálných scénářích může docházet k porušování, obcházení nebo úpravám normálních forem v závislosti na konkrétních potřebách a okolnostech.

Normální formy jsou někdy vnímány spíše jako akademická záležitost. Znalost těchto forem je klíčová pro návrh dobře fungujících databází a jejich aplikace často pomáhá eliminovat nežádoucí anomálie. Avšak jejich striktní dodržování není vždy nezbytně nutné. I přesto je doporučeno v praxi normální formy aktivně využívat, neboť mohou výrazně přispět k dlouhodobé stabilitě a flexibilitě databázového systému.

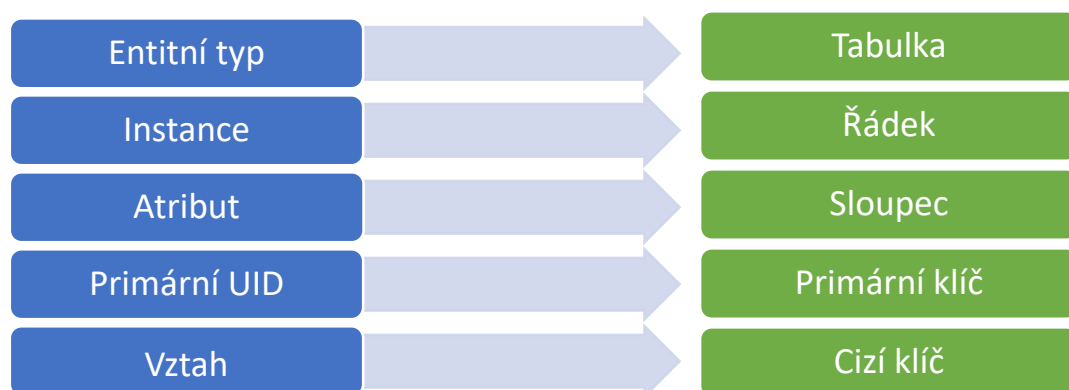
9 Fyzický model

Fyzický model databáze představuje konkrétní implementaci datového modelu v konkrétním databázovém systému (DBMS). Zatímco konceptuální a logický model se zaměřují na strukturu dat a vztahy mezi nimi, fyzický model specifikuje, jak budou data uložena, jaké budou mít datové typy a jak bude optimalizováno jejich zpracování.

9.1 Transformace na fyzický model

Logický model slouží k tomu, aby jasně popsal strukturu dat, jejich vlastnosti a vztahy mezi nimi, aniž bychom se zabývali detaily konkrétního databázového systému. Jakmile ale přecházíme k fyzickému modelu, je nutné tuto strukturu převést do konkrétní podoby odpovídající vybranému DBMS (např. SQL Server).

- **Entitní typy se stávají tabulkami** – Každý entitní typ, který byl v logickém modelu navržen, se ve fyzickém modelu reprezentuje jako tabulka.
- **Atributy se mění na sloupce** – Každý atribut entity se překládá do sloupce tabulky, přičemž mu musí být přiřazen vhodný datový typ (např. číslo, text, datum).
- **Instance entity se stávají řádky tabulky** – Konkrétní záznamy, které entita reprezentuje, jsou ve fyzickém modelu uloženy jako řádky.
- **Identifikátory přecházejí do primárních klíčů** – Každá entita v logickém modelu má jednoznačný identifikátor (UID), který se ve fyzickém modelu definuje jako primární klíč (PRIMARY KEY).
- **Vztahy mezi entitami se realizují pomocí cizích klíčů** – Vztahy, které byly v logickém modelu reprezentovány spojnici mezi entitami, jsou v databázovém systému implementovány jako cizí klíče (FOREIGN KEY), které propojují tabulky mezi sebou.



9.2 Charakteristiky fyzického modelu

- Není již pouhou abstrakcí, ale konkrétní implementací
- Vytvářen pro specifický databázový systém (např. Microsoft SQL Server)
- Obsahuje přesné definice tabulek, indexů, integritních omezení a dalších prvků
- Zohledňuje optimalizaci výkonu a efektivní správu dat

9.3 Komponenty fyzického modelu

- **Entitní typy** – V databázi jsou entity reprezentovány tabulkami. Každá tabulka odpovídá jedné entitě a obsahuje atributy jako sloupce tabulky.
- **Vztahy mezi entitami**
 - 1:1 – Každý záznam v jedné tabulce odpovídá nejvýše jednomu záznamu v druhé tabulce.
 - 1:N – Jeden záznam v tabulce A může být spojen s více záznamy v tabulce B.
 - M:N – Každý záznam v tabulce A může odpovídat více záznamům v tabulce B a naopak.
Tento vztah je obvykle implementován pomocí spojovací (vazební) tabulky.
- **Atributy a datové typy** – Každý atribut entity má definovaný datový typ, který ovlivňuje způsob uložení dat a jejich validaci.
- **Klíče**
 - Primární klíč (PK) – Jedinečný identifikátor záznamu v tabulce
 - Cizí klíč (FK) – Odkazuje na primární klíč jiné tabulky
 - Unikátní klíč (UK) – Zajišťuje jedinečnost hodnot v daném sloupci



Přemýšlejte o transformaci jako o procesu, kdy logický návrh dostává **konkrétní podobu v databázovém systému**, aby mohl být skutečně využíván k ukládání a správě dat.

10 Datové typy v SQL Serveru

Datový typ určuje rozsah a druh hodnot, které mohou být uloženy v daném sloupci tabulky. Správná volba datového typu je klíčová pro:

- **Efektivitu úložiště** – Menší datové typy znamenají nižší nároky na úložný prostor
- **Výkon dotazů** – Optimalizace výpočtů a indexování
- **Integritu dat** – Ochrana proti chybným vstupům

SQL Server poskytuje širokou škálu datových typů, které lze rozdělit do tří hlavních kategorií:

1. **Textové datové typy**
2. **Numerické datové typy**
3. **Typy pro datum a čas**

10.1 Textové datové typy

Datový typ	Maximální délka	Popis
CHAR(n)	8000 znaků	Pevná délka, vždy zabírá n bajtů
NCHAR(n)	4000 znaků	Pevná délka, ukládá Unicode (2 bajty/znak)
VARCHAR(n)	8 000 znaků + 2 bajty	Variabilní délka, šetří místo
NVARCHAR(n)	4 000 znaků + 2 bajty	Variabilní délka s podporou Unicode
VARCHAR(MAX)	2 GB	Variabilní text, vhodný pro velké objemy dat



Použití **VARCHAR(n)** je efektivnější než **CHAR(n)**, pokud se délka textu výrazně liší mezi záznamy. **NVARCHAR** by měl být použit, pokud je nutná podpora Unicode (např. pro mezinárodní aplikace).

10.2 Numerické datové typy

Numerické datové typy můžeme rozdělit na:

- Celá čísla
- Desetinná čísla
- Speciální numerické typy

10.2.1 Celá čísla

Datový typ	Rozsah hodnot	Velikost v bajtech
BIT	0, 1 nebo NULL	1 bit
TINYINT	0 až 255	1 B
SMALLINT	-32 768 až 32 767	2 B
INT	-2 147 483 648 až 2 147 483 647	4 B
BIGINT	-9 223 372 036 854 775 808 až 9 223 372 036 854 775 807	8 B



Používej nejmenší datový typ, který pokryje požadovaný rozsah hodnot – ušetříš místo a zlepšíš výkon dotazů.

10.2.2 Desetinná čísla

Datový typ	Popis	Přesnost
FLOAT	Přibližné číslo	15-16 číslic
DECIMAL(p,s)	Pevná přesnost p = celkový počet číslic s = počet číslic za desetinnou tečkou	1-38 číslic



DECIMAL je vhodnější pro **finanční výpočty**, protože je přesný. FLOAT používá přibližné hodnoty, takže v konečném důsledku může dojít k zaokrouhlovacím chybám.

10.2.3 Typy pro datum a čas

Datový typ	Rozsah hodnot	Přesnost
DATE	0001-01-01 až 9999-12-31	Pouze datum
TIME	00:00:00.000 až 23:59:59.999	Pouze čas
DATETIME	1753-01-01 00:00:00.000 až 9999-12-31 23:59:59.997	Datum a čas
DATETIME2	0001-01-01 00:00:00.000 až 9999-12-31 23:59:59.999	Datum a čas



DATETIME2 je novější a efektivnější než **DATETIME**, doporučuje se pro nové projekty.

11 Úvod do jazyka SQL

SQL (Structured Query Language) je standardizovaný jazyk pro práci s relačními databázemi.

- 1970 – Vývoj relačních databází v IBM
- 1974 – První návrh jazyka **SEQUEL** (Structured English Query Language)
- Později – Přejmenován na **SQL** kvůli ochranné známce jiného produktu
- 1986 – První standard SQL od ANSI
- 1992 – SQL-92, rozšíření o nové funkce
- 2003 – Podpora XML, JSON, analytických funkcí a dalších vylepšení



SQL je dnes podporován všemi hlavními databázovými systémy: **Microsoft SQL Server**, **MySQL**, **PostgreSQL**, **Oracle**, **SQLite** atd.

11.1 Co je SQL?

SQL nabízí široké možnosti práce s databází:

- **Správa databází** – Vytváření a mazání databází
- **Manipulace s daty** – Vkládání, aktualizace a mazání záznamů
- **Dotazování dat** – Získávání a filtrování informací
- **Zabezpečení databáze** – Kontrola uživatelských práv
- **Zajištění integrity** – Definování omezení pro zachování konzistence dat



Výslovnost: SQL se běžně vyslovuje jako "**Es-kjů-el**" nebo "**Síkvl**".

11.2 Podkategorie SQL

Podkategorie	Popis	Příklady příkazů
DDL (Data Definition Language)	Definování databázových objektů	CREATE TABLE, ALTER TABLE, DROP TABLE
DML (Data manipulation Language)	Manipulace s daty	INSERT, UPDATE, DELETE
DQL (Data Query Language)	Získávání dat	SELECT
DCL (Data Control Language)	Řízení přístupu k databázi	GRANT, REVOKE
TCL (Transaction Control Language)	Řízení databázových transakcí	COMMIT, ROLLBACK

11.3 Databázoví klienti

Databázový klient je aplikace nebo nástroj, který umožňuje uživateli **kommunikovat s databází**. Může mít podobu příkazového řádku nebo grafického uživatelského rozhraní (GUI).

11.3.1 Příkazový řádek

Příkazový řádek (CLI – Command Line Interface) umožňuje přímé zadávání SQL příkazů do databázového serveru. Používá se hlavně pro skriptování a automatizaci.

Příklady:

- **sqlcmd** (SQL Server)
- **mysql** (MySQL)
- **psql** (PostgreSQL)

11.3.2 Grafické uživatelské rozhraní

Grafické uživatelské rozhraní (GUI – Graphical User Interface) poskytuje vizuální prostředí pro práci s databázemi. Umožňuje snadnou správu tabulek, záznamů, schémat a práv uživatelů. Příklady populárních databázových klientů:

Klient	Podporované databáze	Popis
SQL Server Management Studio (SSMS)	Microsoft SQL Server	Oficiální nástroj pro správu SQL Serveru, pokročilé funkce pro administraci
Visual Studio Code	SQL Server, MySQL, PostgreSQL, SQLite, Oracle, MongoDB, Redis atd.	Univerzální nástroj, podpora více databází přes rozšíření (např. MSSQL, MySQL, PostgreSQL)
MySQL Workbench	MySQL	Nástroj pro návrh a správu MySQL databází
HeidiSQL	MySQL, MariaDB, PostgreSQL	Lehký a rychlý klient pro práci s databázemi

12 Vytváření tabulek – CREATE TABLE

Příkaz CREATE TABLE patří do podkategorie DDL a používá se k vytvoření nové tabulky v databázi. Syntaxe zahrnuje název tabulky, seznam sloupců s jejich datovými typy, výchozí hodnoty a případná omezení (např. PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT, NOT NULL). Integritní omezení si vysvětlíme v samostatné kapitole.

Základní syntaxe:

```
CREATE TABLE NazevTabulky (  
    NazevAtributu1 DatovyTyp1 OMEZENÍ,  
    NazevAtributu2 DatovyTyp2 OMEZENÍ,  
    ...  
);
```

Příklad pro vytvoření tabulky Zamestnanec:

```
CREATE TABLE Zamestnanec(  
    Id INT,  
    Jmeno NVARCHAR(50),  
    Prijmeni NVARCHAR(50),  
    Plat DECIMAL(10,2),  
    Poznamka NVARCHAR(MAX)  
);
```

Takto vytvořená tabulka by sice fungovala, ale prozatím neobsahuje primární klíč ani jiná omezení či nastavení, která si nyní vysvětlíme a následně se k vytváření tabulky opět vrátíme, vyzbrojeni novými znalostmi.

13 Základy omezení

Integritní omezení (constraints) v SQL určují pravidla, která musí data v tabulce splňovat. Pomáhají zajistit integritu databáze a předcházet nekonzistentním nebo chybným záznamům.

13.1 PRIMARY KEY

Toto omezení určuje hlavní identifikátor záznamu v tabulce. Pokud sloupec označíme omezením PRIMARY KEY (uděláme z něj primární klíč), pak hodnoty sloupce musí být vždy jedinečné a nesmí obsahovat hodnotu NULL. Tabulka může mít jen jeden primární klíč, který však může být složen z více sloupců.

Příklad:

```
CREATE TABLE Zamestnanec(  
    Id INT PRIMARY KEY  
);
```

13.2 IDENTITY

V SQL Serveru je IDENTITY vlastnost sloupce, která automaticky generuje unikátní číselné hodnoty při vkládání nových řádků do tabulky. Takže při vytváření záznamů v tabulce není nutné ručně zadávat hodnoty, o číslování se postará DBMS. IDENTITY se obvykle používá u primárních klíčů. Použít můžeme dvě varianty:

1. IDENTITY se specifikací
2. IDENTITY bez specifikace

13.2.1 IDENTITY se specifikací

U nastavení IDENTITY můžeme zvolit variantu, u níž specifikujeme počáteční hodnotu pro číslování a dále hodnotu pro inkrementaci (krok). Kupříkladu nastavením IDENTITY(1,1) začne SQL Server v daném sloupci číslovat od hodnoty 1 a každá další hodnota poroste o jedničku (1, 2, 3, 4...). Nastavením IDENTITY(5,3) začne SQL Server číslovat od 5 a každá další hodnota poroste o trojku (5, 8, 11, 14...).

V následujícím příkladu bude použito číslování od hodnoty 10 a u každého dalšího záznamu se hodnota zvýší o 3 (10, 13, 16, 19...).

Příklad:

```
CREATE TABLE Zamestnanec(  
    Id INT PRIMARY KEY IDENTITY(10,3)  
);
```

13.2.2 IDENTITY bez specifikace

V běžných situacích se čísluje od jedničky a každý další záznam zvyšuje hodnotu taktéž o jedničku. Pokud chceme provést takovéto nastavení, SQL Server nám nabízí možnost využít IDENTITY bez specifikování hodnot, prostě stačí jen použít slovo IDENTITY a bude to totéž, jako bychom napsali IDENTITY(1,1).

Příklad:

```
CREATE TABLE Zamestnanec(  
    Id INT PRIMARY KEY IDENTITY  
);
```

13.3 FOREIGN KEY

Toto omezení vytváří vazbu mezi dvěma tabulkami (vytváří se tzv. referenční integrita). Hodnota ve sloupci s cizím klíčem musí existovat v primárním klíči jiné tabulky. Tím bude zajištěno, že se do tabulky neuloží neexistující záznamy.

Uveďme si příklad se zaměstnanci a odděleními. V tabulce Zamestnanec vytvoříme jeden sloupec, který bude sloužit jako cizí klíč odkazující do tabulky Oddeleni. Sloupec pro cizí klíč ponese název OddeleniId.

Příklad:

```
CREATE TABLE Zamestnanec (  
    Id INT PRIMARY KEY IDENTITY(1,1),  
    OddeleniId INT,  
    CONSTRAINT FK_Zamestnanec_Oddeleni  
    FOREIGN KEY (OddeleniId)  
    REFERENCES Oddeleni(Id)  
);
```

Při vytváření cizího klíče bývá zvykem pojmenovat toto omezení. Pojmenování se obvykle provádí tak, že nejprve uvedeme FK (foreign key), za ním název tabulky s cizím klíčem, a nakonec název tabulky, do které náš cizí klíč míří. Takže v našem případě FK_Zamestnanec_Oddeleni.



Datový typ sloupce FK musí být stejný jako datový typ sloupce PK v cílové tabulce.

13.4 NOT NULL

Omezení NOT NULL zabraňuje uložení NULL hodnoty do sloupce (hodnota je povinná). V následujícím příkladu toto omezení zajistí, že jméno zaměstnance bude povinné, tedy, že každý zaměstnanec musí mít vyplněno nějaké jméno.

Příklad:

```
CREATE TABLE Zamestnanec(  
    Jmeno NVARCHAR(50) NOT NULL  
);
```

13.5 UNIQUE

Omezení typu UNIQUE zajišťuje, že hodnoty ve sloupci nebudou duplicitní. V následujícím příkladu je provedeno použití UNIQUE, které zajistí, že žádné dva záznamy v tabulce nemohou mít stejnou emailovou adresu.

Příklad:

```
CREATE TABLE Zamestnanec(  
    Email VARCHAR(100) UNIQUE  
);
```

13.6 CHECK

Toto omezení definuje podmínku, kterou musí hodnota vždy splňovat. V následujícím příkladu je aplikováno pravidlo, které kontroluje, že každý zaměstnanec má nejméně 18 let.

Příklad:

```
CREATE TABLE Zamestnanec (  
    Vek INT CHECK (Vek >= 18)  
);
```

13.7 DEFAULT

Klauzule DEFAULT technicky nepatří mezi omezení, protože nic neomezuje, ale obvykle je vysvětlována společně s omezeními, s nimiž se v rámci práce s tabulkami funkčně doplňuje. Tato klauzule doplní hodnotu, pokud není při vkládání nového záznamu zadána.

V následujícím příkladu je klauzule DEFAULT použita u sloupce Poznamka. Pokud u nového záznamu pro zaměstnance nebude zadána žádná jiná hodnota, použije se hodnota klauzule DEFAULT.

Příklad:

```
CREATE TABLE Zamestnanec (  
    Poznamka NVARCHAR(100) DEFAULT N'Nový zaměstnanec'  
);
```

14 Tvorba tabulek s omezeními

Nyní, když jsme vyzbrojeni novými znalostmi, můžeme si uvést několik příkladů, kterými vytvoříme plnohodnotné tabulky s omezením i nastavením. Tato kapitola je rozdělena na dvě podkapitoly. V první si ukážeme, jak lze do tabulky přidat omezení jako PRIMARY KEY, NOT NULL, CHECK, DEFAULT, UNIQUE a IDENTITY. V druhé části se zaměříme na to, jak vytvořit cizí klíče a nastavit referenční integritu mezi tabulkami.

14.1 Integritní omezení v rámci tabulky

První příklad prezentuje tabulku Zaměstnanec se sloupci id, jméno, příjmení, platem a poznámkou. Hodnotu sloupce id budeme chtít automaticky zvyšovat o jedničku, jméno a příjmení bude vždy povinné a plat musí být větší než nula. Poznámka bude nepovinná s výchozí hodnotou v případě, že ji uživatel nevyplní.

```
CREATE TABLE Zaměstnanec (  
    Id INT PRIMARY KEY IDENTITY,  
    Jmeno NVARCHAR(50) NOT NULL,  
    Prijmeni NVARCHAR(50) NOT NULL,  
    Plat DECIMAL(10,2) CHECK (Plat > 0),  
    Poznámka NVARCHAR(100) DEFAULT N'Nový zaměstnanec'  
);
```

V následujícím příkladu vytvoříme tabulku Produkt, která bude uchovávat informace o skladových položkách. Každý produkt bude mít název, cenu, množství a volitelný popis. Cenu omezíme tak, aby byla větší než nula, nebudeme přeci zboží rozdávat zdarma.

```
CREATE TABLE Produkt (  
    Id INT PRIMARY KEY IDENTITY,  
    Nazev NVARCHAR(100) NOT NULL,  
    Cena DECIMAL(10,2) CHECK (Cena > 0),  
    Mnozstvi INT NOT NULL,  
    Popis NVARCHAR(255) DEFAULT N'Bez popisu'  
);
```

Zkusme ještě jeden příklad. Tabulka Kurz bude reprezentovat školení nebo kurzy. Každý kurz bude mít název, datum konání a volitelný počet volných míst. Počet míst musí při vkládání nového kurzu do databáze být větší než nula, jinak by takový nový kurz neměl smysl. Pokud uživatel neuvede datum, nastavíme ho automaticky například na 1. září 2025, aby nám nezůstalo nevyplněno.

```
CREATE TABLE Kurz (  
    Id INT PRIMARY KEY IDENTITY,  
    Nazev NVARCHAR(100) NOT NULL,  
    Datum DATE DEFAULT '2025-09-01',  
    VolnaMista INT DEFAULT 0 CHECK (VolnaMista >= 0)  
);
```

14.2 Vytváření vazeb mezi tabulkami (FK)

Již jsme si vysvětlili, že toto omezení vytváří vazbu mezi dvěma tabulkami (vytváří se tzv. referenční integrita). Cizí klíč (FOREIGN KEY) propojuje dvě tabulky – říká, že jedna tabulka „odkazuje“ na záznam v jiné tabulce. Díky tomu databáze zkontroluje, že například zaměstnanec je přiřazen ke skutečně existujícímu oddělení. Jinými slovy: pokud se v tabulce objeví hodnota cizího klíče, databáze nejprve ověří, že tato hodnota už existuje jako primární klíč v jiné tabulce. Pokud neexistuje, záznam neuloží. Databáze nedovolí přiřadit zaměstnance k neexistujícímu oddělení – cizí klíč hlídá, že vše sedí.

Pojďme si vyzkoušet první příklad. Každý zaměstnanec pracuje v nějakém oddělení. Sloupec OddeleniId v tabulce Zamestnanec odkazuje na Id v tabulce Oddeleni.

```

CREATE TABLE Oddeleni (
    Id INT PRIMARY KEY IDENTITY,
    Nazev NVARCHAR(50) NOT NULL
);

CREATE TABLE Zamestnanec (
    Id INT PRIMARY KEY IDENTITY,
    Jmeno NVARCHAR(50) NOT NULL,
    Prijmeni NVARCHAR(50) NOT NULL,
    OddeleniId INT NOT NULL,
    CONSTRAINT FK_Zamestnanec_Oddeleni
        FOREIGN KEY (OddeleniId)
        REFERENCES Oddeleni(Id)
);

```

Dalším příkladem jsou tabulky Student a Třída. Každý student je zařazen do určité třídy. Cizí klíč TridaId v tabulce Student odkazuje na tabulku Trida.

```

CREATE TABLE Trida (
    Id INT PRIMARY KEY IDENTITY,
    Zkratka NVARCHAR(10) NOT NULL
);

CREATE TABLE Student (
    Id INT PRIMARY KEY IDENTITY,
    Jmeno NVARCHAR(50) NOT NULL,
    Prijmeni NVARCHAR(50) NOT NULL,
    TridaId INT NOT NULL, -- Povinný cizí klíč
    CONSTRAINT FK_Student_Trida
        FOREIGN KEY (TridaId)
        REFERENCES Trida(Id)
);

```

Ve třetím příkladu si ukážeme tabulky Zákazníka a objednávky. Každá objednávka musí být spojena s nějakým zákazníkem. Sloupec ZakaznikId odkazuje na Id zákazníka.

```
CREATE TABLE Zakaznik (  
    Id INT PRIMARY KEY IDENTITY,  
    Jmeno NVARCHAR(100) NOT NULL  
);  
  
CREATE TABLE Objednavka (  
    Id INT PRIMARY KEY IDENTITY,  
    Datum DATE NOT NULL,  
    ZakaznikId INT, -- Volitelný cizí klíč  
    CONSTRAINT FK_Objednavka_Zakaznik  
        FOREIGN KEY (ZakaznikId)  
        REFERENCES Zakaznik(Id)  
);
```

V posledním příkladu si ukážeme, že hodnota cizího klíče nemusí být vždy povinná. Jinými slovy, záznam nemusí být vždy nutně ve vztahu se záznamem z cizí tabulky. Příklad s knihami a čtenáři nám k tomuto skvěle poslouží.

Tabulka Kniha obsahuje všechny knihy v knihovně. Pokud je kniha vypůjčená, bude mít ve sloupci CtenarId uvedeno Id čtenáře z tabulky Ctenar. Pokud je kniha volná, tento sloupec zůstane prázdný (NULL).

```
CREATE TABLE Ctenar (  
    Id INT PRIMARY KEY IDENTITY,  
    Jmeno NVARCHAR(100) NOT NULL  
);  
  
CREATE TABLE Kniha (  
    Id INT PRIMARY KEY IDENTITY,  
    Nazev NVARCHAR(100) NOT NULL,  
    CtenarId INT, -- Volitelný cizí klíč  
    CONSTRAINT FK_Kniha_Ctenar  
        FOREIGN KEY (VypujcenoKym)  
        REFERENCES Ctenar(Id)  
);
```

15 Úprava tabulky – ALTER TABLE

Příkaz ALTER TABLE slouží k úpravě již existujících tabulek v databázi. Pomocí tohoto příkazu můžeme například:

- přidat nový sloupec
- změnit datový typ existujícího sloupce
- odebrat sloupec

Díky tomu není nutné tabulku mazat a vytvářet znovu, pokud potřebujeme provést jen menší úpravy. Příkaz ALTER TABLE se často používá v praxi při vývoji aplikací, kdy se datový model postupně vyvíjí a upravuje podle potřeb.

V dalších příkladech si ukážeme, jak konkrétně tabulku rozšířit, omezit nebo změnit pomocí ALTER TABLE.

15.1 Přidání sloupce

Při návrhu databáze se může stát, že si na začátku nevzpomeneme na všechny potřebné sloupce. Příkaz ALTER TABLE nám umožní dodatečně rozšířit strukturu tabulky – konkrétně pomocí klauzule ADD můžeme přidat nový sloupec.

Přidaný sloupec je obvykle prázdný (NULL) pro již existující záznamy, dokud mu nenastavíme nějaké hodnoty. Pokud k němu rovnou přidáme i DEFAULT, můžeme doplnit výchozí hodnoty automaticky.

```
-- Přidání sloupce pro poznámku do tabulky Zamestnanec  
ALTER TABLE Zamestnanec  
ADD Poznamka NVARCHAR(MAX);
```

```
-- Přidání datumu registrace do tabulky Ctenar  
ALTER TABLE Ctenar  
ADD DatumRegistrace DATE;
```



Všimněme si, že za klíčovými slovy ALTER TABLE se již nenacházejí kulaté závorky, jako je tomu u příkazu CREATE TABLE.

15.2 Úprava existujícího sloupce

Databázová struktura se vyvíjí společně s potřebami systému. Pokud například zjistíme, že délka sloupce nestačí, nebo že sloupec nemá být povinný, můžeme jeho definici změnit pomocí klauzule `ALTER COLUMN`.

Tímto způsobem lze upravit datový typ, délku textu nebo přidat/odebrat omezení typu `NOT NULL`. Je důležité dát pozor na to, aby změna typu neporušila stávající data – například přechod z `INT` na `NVARCHAR` je technicky možný, ale může znamenat ztrátu smyslu dat.

```
-- Změna typu a omezení u sloupce Jmeno v tabulce Zamestnanec
ALTER TABLE Zamestnanec
ALTER COLUMN Jmeno NVARCHAR(50) NOT NULL;
```

```
-- Změna typu sloupce Cena v tabulce Produkt na větší přesnost
ALTER TABLE Produkt
ALTER COLUMN Cena DECIMAL(12,2);
```

15.3 Vymazání sloupce

Pokud sloupec již není potřeba, můžeme ho z tabulky úplně odstranit pomocí příkazu `DROP COLUMN`. Tímto způsobem se z databáze odstraní nejen samotná struktura, ale i veškerá data, která ve sloupci byla.

Před odstraněním sloupce je dobré zvážit, zda se jeho data nepoužívají v dotazech nebo vazbách. Pokud by sloupec obsahoval cizí klíč nebo jiné omezení, může být nutné nejprve tato omezení zrušit.

```
-- Odstranění sloupce Prijmeni z tabulky Zamestnanec  
ALTER TABLE Zamestnanec  
DROP COLUMN Prijmeni;
```

```
-- Odstranění sloupce Popis z tabulky Produkt  
ALTER TABLE Produkt  
DROP COLUMN Popis;
```

15.4 Přejmenování sloupce

Název sloupce můžeme změnit například v případě, že původní název nebyl výstižný, nebo se změnila terminologie aplikace. K tomu slouží systémová procedura **sp_rename**, kterou poskytuje SQL Server.

Při přejmenování musíme vždy zadat název tabulky i sloupce ve formátu 'Tabulka.Sloupec' a nový název sloupce. Tento postup neovlivní samotná data – pouze upraví popisný název sloupce.

```
-- Přejmenování sloupce Telefon na Mobil v tabulce Zamestnanec  
EXEC sp_rename 'Zamestnanec.Telefon', 'Mobil', 'COLUMN';
```

```
-- Přejmenování sloupce Cena na CenaKc v tabulce Produkt  
EXEC sp_rename 'Produkt.Cena', 'CenaKc', 'COLUMN';
```

15.5 Přejmenování tabulky

Pokud se změní význam tabulky nebo chceme lépe vyjádřit její účel, můžeme tabulku přejmenovat pomocí stejné procedury **sp_rename**. Stačí uvést původní název tabulky a nový název.

Díky tomu nemusíme tabulku mazat a vytvářet znovu, čímž zachováme všechna data i vazby. Přejmenování je čistě kosmetická změna názvu, ale může zlepšit čitelnost a údržbu databáze.

```
-- Přejmenování tabulky Zamestnanec na Pracant  
EXEC sp_rename 'Zamestnanec', 'Pracant';
```

```
-- Přejmenování tabulky Objednavka na Nakup  
EXEC sp_rename 'Objednavka', 'Nakup';
```

15.6 Vymazání tabulky

Pokud tabulku již v databázi nepotřebujeme, můžeme ji úplně odstranit pomocí příkazu **DROP TABLE**. Tím se trvale smaže celá tabulka včetně její struktury i dat.

Tato operace je nevratná – jakmile tabulku smažeme, nelze ji jednoduše obnovit. Proto se doporučuje před mazáním důležitých tabulek vytvořit zálohu, nebo se ujistit, že tabulka skutečně není nikde používána.

```
-- Smazání tabulky Zamestnanec  
DROP TABLE Zamestnanec;
```

```
-- Smazání tabulky DocasnaData  
DROP TABLE DocasnaData;
```

16 Vkládání dat – INSERT

Příkaz INSERT slouží k vložení nového záznamu (řádku) do existující tabulky. Patří mezi základní příkazy jazyka DML (Data Manipulation Language), tedy příkazy, které přímo pracují s obsahem databázových tabulek.

Pomocí INSERT můžeme do tabulky přidat jeden nebo více záznamů najednou. Při vkládání musíme dodržet pořadí sloupců a datové typy, aby byla data uložena správně. Hodnoty, které nejsou uvedeny, zůstanou prázdné (NULL), nebo se použije výchozí hodnota, pokud je nastavená (DEFAULT).

Unicode prefix

Ještě, než začneme, je důležité nezapomenout na jednu důležitou věc, která je specifická pro Microsoft SQL Server. Tou věcí je tzv. **Unicode prefix**. V SQL Serveru se Unicode prefix **N** (jako National) používá pro označení Unicode řetězce. To znamená, že text je uložen v kódování, které podporuje všechny znaky různých jazyků – včetně češtiny, azbuky, čínštiny apod.

Bez Unicode prefixu **N** by SQL Server předpokládal tzv. ANSI (nebo ASCII) řetězec – a mohl by:

- ztratit české znaky (ě, š, č, ž...)
- zobrazit text chybně (např. „????“)
- uložit nesmyslná data

Proto vždy, když vkládáme textové hodnoty do sloupců typu NVARCHAR (nebo NCHAR), **je potřeba psát N před řetězec**.

16.1 Syntaxe

Příkaz INSERT se zapisuje tak, že nejprve uvedeme klíčová slova INSERT INTO, následovaná názvem tabulky, do které chceme data vložit. Za název tabulky v závorkách uvedeme názvy sloupců, kterým chceme přiřadit hodnoty – pořadí sloupců musí odpovídat pořadí hodnot, které následují za klauzulí VALUES. V ní zapíšeme jednotlivé hodnoty také do závorek. Při vkládání více řádků najednou se každá sada hodnot uzavírá do vlastních závorek a odděluje čárkami.

16.2 Vložení jednoho řádku

Dodržením základní syntaxe docílíme vložení jednoho nového řádku do tabulky.

```
-- Vložení nového zaměstnance do tabulky Zamestnanec
INSERT INTO Zamestnanec (Jmeno, Prijmeni, Plat)
VALUES (N'Anna', N'Svobodová', 35000);
```

```
-- Vložení jednoho čtenáře do tabulky Ctenar
INSERT INTO Ctenar (Jmeno, Prijmeni, DatumRegistrace)
VALUES (N'Martin', N'Dvořák', '2024-09-01');
```

16.3 Vložení více řádků

Nyní si ukážeme způsob, jakým se dá vložit více záznamů pomocí jednoho příkazu INSERT INTO. Začátek je stejný, rozdíl nastává za klauzulí **VALUES**, kdy za uzavírací závorkou prvního záznamu nepíšeme středník, ale čárku a poté pokračujeme dalšími řádky. Středník nakonec uvedeme až za poslední kulatou závorkou.

```
-- Vložení více zaměstnanců najednou
INSERT INTO Zamestnanec (Jmeno, Prijmeni, Plat)
VALUES
    (N'Petr', N'Nový', 30000),
    (N'Jan', N'Starý', 32000),
    (N'Lucie', N'Mladá', 31000);
```

```
-- Vložení více produktů do tabulky Produkt
INSERT INTO Produkt (Nazev, Cena)
VALUES
    (N'USB Flash 64 GB', 399),
    (N'Bezdrátová myš', 499),
    (N'Klávesnice', 899);
```

V prvních kulatých závorkách s výčtem atributů nemusíme vždy nutně uvádět všechny atributy. Všimněme si například toho, že ve všech našich příkazech chybí třeba atribut **Id**. Pokud je atribut **Id** v tabulce nastaven pomocí **IDENTITY** na automatické číslování, pak jej nemusíme uvádět, protože o vyplnění jeho hodnoty se za nás postará **SQL Server**.

Podobné je to i s jinými atributy, které jsou například volitelné (nemusí se vyplňovat). Pokud je zde neuvedeme a nevyplníme, pak bude jejich hodnota **NULL** nebo budou automaticky vyplněny výchozí hodnotou, pokud ji mají v rámci tabulky nastavenou pomocí klíčového slova **DEFAULT**.

17 Úprava dat – UPDATE

Příkaz UPDATE slouží ke změně (aktualizaci) dat, která už v tabulce existují. Umožňuje upravit hodnoty v jednom nebo více sloupcích vybraných řádků. V praxi se často používá k opravám údajů, přepisům chybných hodnot nebo k aktualizaci záznamu.

Abychom aktualizovali správný řádek, používá se s příkazem UPDATE také podmínka WHERE. Ta určuje, které řádky se mají změnit. Pokud podmínku vynecháme, změna se provede ve všech řádcích tabulky – proto je důležité s příkazem UPDATE pracovat opatrně. Podmínka se dá nastavovat vůči libovolnému sloupci, ale pokud potřebujeme zacílit na konkrétní jeden záznam, pak je vždy největší jistotou využít jeho Id, tedy **primární klíč**.

17.1 Syntaxe

Příkaz UPDATE slouží k úpravě existujících záznamů v databázové tabulce. Začíná názvem tabulky, kterou chceme upravit, a pokračuje klauzulí SET, kde uvádíme jednotlivé změny (co a na jakou hodnotu chceme přepsat).

```
UPDATE NázevTabulky
SET Sloupec1 = NováHodnota1,
    Sloupec2 = NováHodnota2,
    ...
WHERE Podmínka;
```

17.2 Aktualizace jednoho atributu

Pojďme se podívat na první příklad, v němž změníme jméno u zaměstnance, který má v atributu Id hodnotu 4.

```
-- Změna jména zaměstnance s ID 4
UPDATE Zamestnanec
SET Jmeno = N'Zdislav'
WHERE Id = 4;
```

V dalším příkladu změníme název produktu u záznamu s id 2.

```
-- Úprava názvu produktu podle jeho ID
UPDATE Produkt
SET Nazev = N'Bezdrátová myš'
WHERE Id = 2;
```

17.3 Aktualizace více atributů najednou

V praxi je běžné měnit více než jeden atribut u zvoleného záznamu. V následujícím příkladu si uvedeme změnu dvou atributů v rámci jednoho příkazu UPDATE.

```
-- Úprava jména a příjmení zaměstnance
UPDATE Zamestnanec
SET Jmeno = N'Zdislav', Prijmeni = N'Uzdistál'
WHERE Id = 4;
```

V dalším příkladu změníme rovnou 4 atributy, což není v praxi nic zvláštního. Pro lepší čitelnost si můžeme jednotlivé atributy klidně rozepsat na jednotlivé řádky.

```
-- Úprava zaměstnance s ID 7
UPDATE Zamestnanec
SET
    Jmeno = N'Ivana',
    Prijmeni = N'Hrubá',
    Plat = 39000,
    Poznamka = N'Aktualizováno dle HR'
WHERE Id = 7;
```

Ukažme si ještě jeden příklad, u nějž **vynecháme klauzuli WHERE**, čímž způsobíme hromadnou úpravu pro všechny řádky tabulky. V tomto případě se firma rozhodla, že **veškeré zboží** na skladě označí jako zboží ve výprodeji.

```
UPDATE Produkt
```

```
SET Popis = N'Zboží ve výprodeji';
```

Tento UPDATE způsobí nastavení popisku u všech záznamů v tabulce.

18 Mazání dat – DELETE

Příkaz **DELETE** slouží k odstranění jednoho nebo více řádků z databázové tabulky. Na rozdíl od příkazů **INSERT** a **UPDATE**, které přidávají nebo upravují data, **DELETE** trvale maže existující záznamy. Při použití tohoto příkazu je důležité být velmi opatrný, protože smazaná data nelze jednoduše vrátit zpět – pokud databáze nemá nastaveno zálohování nebo historii změn, data jsou nenávratně pryč.

Mazání se obvykle provádí pomocí podmínky **WHERE**, která určuje, které řádky mají být odstraněny. Pokud tuto podmínku vynecháme, smažou se všechny záznamy v tabulce. To se sice může někdy hodit, ale zároveň představuje velké riziko chyby – proto je vhodné **DELETE** bez **WHERE** vždy důkladně zvážit.

18.1 Syntaxe

Příkaz **DELETE** se zapisuje velmi jednoduše. Nejprve uvedeme klíčová slova **DELETE FROM**, po kterých následuje název tabulky, ze které chceme odstranit záznamy. Dále může následovat klauzule **WHERE**, která určuje podmínku mazání – tedy které konkrétní řádky mají být odstraněny. Pokud podmínku **WHERE** vynecháme, příkaz **DELETE** odstraní **všechny řádky** z dané tabulky.

```
DELETE FROM NázevTabulky  
WHERE Podmínka;
```

18.2 Smazání jednoho řádku

Příkaz se v následujícím příkladu postará o smazání všech záznamů, které mají ID větší než 20.

```
DELETE FROM Zamestnanec  
WHERE Id > 20;
```

18.3 Smazání všech řádků

Obdobně jako v případě příkazu **UPDATE**, i u mazání bude vynechání klauzule **WHERE** znamenat aplikaci příkazu na všechny řádky tabulky.

```
DELETE FROM Zamestnanec;
```

18.4 Promazání tabulky

Pokud potřebujeme **rychle a hromadně smazat všechny řádky** z tabulky, můžeme použít také příkaz **TRUNCATE TABLE**. Tento příkaz:

- smaže všechna data z tabulky
- zároveň resetuje IDENTITY (např. automatické číslování ID)
- je rychlejší než DELETE

Příklad:

```
TRUNCATE TABLE Zamestnanec;
```

19 Základní výběr dat – SELECT

Příkaz **SELECT** slouží k výběru dat z databázové tabulky. Je to nejčastěji používaný příkaz v databázích, protože nám umožňuje „nahlédnout“ do uložených dat a zjistit, co se v tabulce nachází. Pomocí **SELECT** si můžeme vybrat data z konkrétních sloupců, které nás zajímají, nebo jednoduše zobrazit všechna dostupná data v tabulce.

19.1 Syntaxe

Každý dotaz začíná klíčovým slovem **SELECT**, po kterém následuje seznam sloupců (nebo hvězdička *, pokud chceme zobrazit všechny). Klíčové slovo **FROM** určuje, ze které tabulky se data mají číst. Výsledkem dotazu je tabulka (výpis), kterou nám SQL Server zobrazí.

```
SELECT Sloupec1, Sloupec2, ...  
FROM NázevTabulky;
```

19.2 Výběr všech sloupců a všech záznamů

V tomto příkladu si ukážeme, jak jednoduše zobrazit **všechna data** z tabulky – tedy všechny sloupce a všechny řádky. Použijeme hvězdičku (*), která zastupuje všechny sloupce tabulky. Výstupem bude úplný obsah tabulky.

```
SELECT * FROM Zamestnanec;
```

19.3 Výběr zvolených sloupců

Nyní si ukážeme, jak zobrazit pouze konkrétní sloupce, například jméno, příjmení a plat zaměstnanců. Ostatní sloupce budou ve výstupu vynechány. Výstupem bude tabulka se třemi sloupci: Jmeno, Prijmeni, Plat.

```
SELECT Jmeno, Prijmeni, Plat  
FROM Zamestnanec;
```

19.4 Výběr konkrétních sloupců s podmínkou

V tomto příkladu použijeme výběr konkrétních sloupců a zároveň přidáme podmínku, že nás zajímá pouze zaměstnanec s ID číslem 7. Podmínka se zapisuje pomocí klauzule WHERE, jako u předchozích kapitol. Výstupem bude jeden řádek se jménem, příjmením a platem zaměstnance, jehož ID je rovno 7.

```
SELECT Jmeno, Prijmeni, Plat  
FROM Zamestnanec  
WHERE Id = 7;
```



V rámci psaní SQL skriptů si můžeme pomáhat libovolným odsazováním textu pomocí tabulátoru nebo enteru. SQL Server to přežije a nám to může zvýšit čitelnost kódu, abychom se v něm neztráceli, především u delších skriptů.

Stejně tak je možno pro milovníky sebemrskání, zapisovat celý kód na jediný řádek :-)

20 Alias

Při práci s dotazy často nastane situace, kdy potřebujeme výstupní sloupce pojmenovat jinak, než jsou pojmenovány v databázi. Například chceme výsledek zobrazit v češtině, skrýt technický název sloupce, nebo nahradit výstupní název funkce jako COUNT(*) něčím srozumitelnějším. V takových případech použijeme **alias** – dočasné jméno, které sloupci přiřadíme pouze v rámci aktuálního dotazu.

Alias můžeme použít také pro tabulky, a to zejména v případech, kdy pracujeme s více tabulkami současně. Místo opakovaného psaní dlouhého názvu tabulky použijeme zkrácený alias a tím výrazně zpřehledníme celý SQL dotaz. Alias se zapisuje pomocí klíčového slova **AS**, ale jeho použití není povinné – lze jej vynechat, avšak doporučuje se ho používat kvůli čitelnosti kódu.

Alias nijak nemění skutečná jména sloupců nebo tabulek v databázi – je to pouze dočasné přejmenování v rámci jednoho dotazu, které pomáhá lépe porozumět výstupu. Je to běžná a velmi užitečná součást jazyka SQL.

20.1 Alias pro sloupec

Sloupec v dotazu můžeme přejmenovat tak, že za něj napíšeme klíčové slovo **AS** a následně nový název:

```
SELECT Sloupec AS NovyNazev  
FROM Tabulka;
```

```
SELECT FirstName AS KrestniJmeno  
FROM Zamestnanec;
```

```
SELECT CONCAT(Jmeno, ' ', Prijmeni) AS [Celé jméno]  
FROM Zamestnanec;
```



Pokud je alias složen z více slov s použitím mezer, pak se vkládá do hranatých závorek.

20.2 Alias pro tabulku

Alias tabulky se zapisuje obdobně. Po názvu tabulky v části FROM (nebo při spojení více tabulek) uvedeme klíčové slovo **AS** a nový krátký název tabulky:

```
SELECT Z.Jmeno  
FROM Zamestnanec AS Z;
```

Nyní můžeme v celém dotazu místo Zamestnanec psát pouze T, což se hodí zejména u spojování více tabulek (JOIN) nebo u dlouhých názvů. Opět platí, že AS je volitelné:

```
SELECT * FROM Zamestnanec Z;
```



V praxi se často klíčové slovo **AS** neuvádí pro alias tabulek, ale u aliasu jednotlivých sloupců pro zvýšení přehlednosti zpravidla uveden bývá.

21 Logické operátory

Při výběru dat často potřebujeme zadat více podmínek najednou – například „zaměstnanec se jmenuje Jan a pracuje v oddělení 2“, nebo „je z Prahy, nebo má plat vyšší než 40 000 Kč“. Abychom takové kombinace mohli zapisovat, používáme logické operátory, které nám umožní spojovat nebo negovat jednotlivé podmínky. Stejně jako v programování.

Logické operátory pracují s pravdivostní hodnotou – výsledkem výrazu je buď TRUE (pravda), nebo FALSE (nepravda). Díky tomu můžeme snadno filtrovat výsledky podle složitějších pravidel.

21.1 Operátor AND

Operátor **AND** slouží ke spojení dvou nebo více podmínek, které **musí být všechny pravdivé**. Používá se například pro výběr zaměstnance podle více kritérií. Následující dotaz vrátí pouze zaměstnance, který se jmenuje Jan Novák.

```
SELECT *  
FROM Zamestnanec  
WHERE Jmeno = N'Jan' AND Prijmeni = N'Novák';
```

21.2 Operátor OR

Pomocí operátoru **OR** můžeme vybrat řádky, které **splňují alespoň jednu z uvedených podmínek**. Je vhodný například pro výběr více variant. Následující dotaz vrátí všechny zaměstnance, kteří se jmenují Jan nebo se jmenují Novák (nebo obojí).

```
SELECT *  
FROM Zamestnanec  
WHERE Jmeno = N'Jan' OR Prijmeni = N'Novák';
```

21.3 Negace

Operátor **NOT** říká, že nechceme řádky, které splňují danou podmínku. Jeho použití je časté u výběrů „vše kromě“. Následující dotaz vrátí všechny zaměstnance kromě těch, kteří se jmenují Jan.

```
SELECT *  
FROM Zamestnanec  
WHERE NOT (Jmeno = N'Jan');
```

21.4 Práce s neznámými hodnotami

V databázích může být hodnota ve sloupci neznámá – tedy NULL. Porovnávat NULL pomocí = nefunguje, protože NULL neznamená „prázdné“ ale „nevíme“. Proto musíme použít speciální operátory **IS NULL** nebo **IS NOT NULL**.

Tento dotaz vrátí všechny zaměstnance, kteří nemají žádnou poznámku zadanou.

```
SELECT *  
FROM Zamestnanec  
WHERE Poznamka IS NULL;
```

A tento zase všechny zaměstnance, kteří poznámku vyplněnou mají (je v ní nějaká hodnota).

```
SELECT *  
FROM Zamestnanec  
WHERE Poznamka IS NOT NULL;
```

22 Omezení výsledků při výběru

Při práci s databázemi většinou nechceme zobrazit úplně všechna data – chceme spíše vybrat jen ta, která nás zajímají. Příkaz SELECT proto často doplňujeme o různá omezení, která nám pomáhají filtrovat, omezit nebo zpřehlednit výstup. SQL nabízí řadu nástrojů, jak výběr zúžit: klauzuli WHERE pro filtrování, kterou už známe. Klauzuli TOP pro omezení počtu výsledků, IN a BETWEEN pro práci s více hodnotami nebo rozsahy a DISTINCT pro odstranění duplicit.

Tato omezení nejen zlepšují výkon při práci s rozsáhlými tabulkami, ale také pomáhají uživateli zobrazit pouze ty údaje, které skutečně potřebuje.

22.1 Filtrování pomocí WHERE

Přestože jsme s klauzulí WHERE již pracovali, pro úplnost je zde uvedena také. Klauzule WHERE slouží k tomu, abychom vybrali pouze ty řádky, které splňují určitou podmínku. Bez ní by se zobrazila všechna data v tabulce. WHERE je jeden z nejčastěji používaných nástrojů v SQL. V tomto případě se bude jednat o filtraci dat na jeden řádek zaměstnance s ID 7.

```
SELECT *  
FROM Zamestnanec  
WHERE Id = 7;
```

22.2 Omezení počtu výsledků pomocí TOP

Klauzule TOP slouží k omezení počtu záznamů, které nám dotaz vrátí. Nejčastěji se používá v testovacích dotazech nebo při výpisu "nejnovějších" nebo "nejlevnějších" položek. Následující příklady zobrazí prvních 100 záznamů z tabulky.

```
SELECT TOP 100 * FROM Zamestnanec;
```

```
SELECT TOP 100 Jmeno, Prijmeni FROM Zamestnanec;
```

22.3 Výběr z více možností pomocí IN

Když potřebujeme zjistit, jestli hodnota patří mezi několik zadaných možností, použijeme operátor **IN**. Místo několika OR podmínek tak můžeme psát přehledněji a kratším způsobem. Následující příklad zobrazí zaměstnance, kteří spadají pod oddělení 1, 2, 8 nebo 14.

```
SELECT *  
FROM Zamestnanec  
WHERE OddeleniId IN (1, 2, 8, 14);
```

22.4 Omezení hodnot v rozsahu – BETWEEN

Pomocí operátoru **BETWEEN** můžeme snadno definovat, že hodnota má být v určitém číselném nebo datumovém rozsahu. Obě hranice jsou zahrnuty (včetně). Následující příklad vrátí všechny zaměstnance, jejichž mzda spadá do daného rozsahu, včetně krajních hodnot.

```
SELECT *  
FROM Zamestnanec  
WHERE Mzda BETWEEN 30000 AND 45000;
```

22.5 Odstranění duplicitních řádků – DISTINCT

Klauzule **DISTINCT** zajistí, že se ve výsledku zobrazí pouze **unikátní** hodnoty. Je vhodná například při hledání všech existujících hodnot v určitém sloupci (např. seznam měst, pozic apod.) **bez opakování**. V následujícím příklad se každá pozice zobrazí pouze jednou – i když v tabulce existuje vícekrát.

```
SELECT DISTINCT Pozice  
FROM Zamestnanec;
```



V textu se objevují slova **klauzule** a **operátor**. Jaký je mezi nimi vlastně rozdíl?
Klauzule jsou části SQL syntaxe, které určují strukturu dotazu (WHERE, TOP, DISTINCT...)
Operátory jsou nástroje uvnitř klauzulí, které provádějí porovnání nebo logickou kontrolu (např. IN, BETWEEN, LIKE, AND...)

23 Řazení výsledků – ORDER BY

Při výběru dat z tabulky často nestačí jen zobrazit určité záznamy – potřebujeme, aby byly seřazeny podle určitého kritéria. Například chceme výpis zaměstnanců podle příjmení, nebo seznam objednávek od nejnovějších po nejstarší. K tomu slouží klauzule **ORDER BY**.

ORDER BY se zapisuje na konec dotazu a říká databázi, jak má výsledky seřadit. Můžeme řadit podle jednoho nebo více sloupců a určit, zda má být řazení **vzestupné (ASC)** nebo **sestupné (DESC)**. Pokud typ řazení neuvedeme, použije se vzestupné (ASC) jako výchozí.

23.1 Řazení podle jednoho sloupce

V tomto příkladu seřadíme zaměstnance podle příjmení. Výsledkem bude seznam od A do Z. Používáme výchozí vzestupné řazení (ASC), které není třeba explicitně uvádět.

```
SELECT *  
FROM Zamestnanec  
ORDER BY Prijmeni;
```

23.2 Řazení podle více sloupců

Nyní budeme řadit nejprve podle příjmení (vzestupně), a v případě shody ještě podle jména (sestupně). Výstup bude seřazen od A do Z podle příjmení, a u shodných příjmení budou jména seřazena opačně – od Z do A.

```
SELECT *  
FROM Zamestnanec  
ORDER BY Prijmeni ASC, Jmeno DESC;
```



Velmi často se studentům pletou pojmy vzestupně a sestupně. **Vzestupné** řazení znamená řazení od A do Z, od 1 do 10, od 2025-01-01 do 2025-12-31 atd. **Sestupné** řazení je přesně naopak.

24 Funkce v SQL

V SQL můžeme kromě klasického výběru a filtrování dat také přímo v dotazu počítat, upravovat nebo zpracovávat hodnoty. Právě k tomu slouží funkce.

SQL funkce jsou předdefinované nástroje, které můžeme využít například v příkazu SELECT, a které vracejí výsledek na základě vstupní hodnoty. Fungují podobně jako vzorce v Excelu – zadáte hodnotu nebo sloupec a funkce z něj něco „vytáhne“, „vypočítá“ nebo „upraví“.

24.1 Textové a znakové funkce

V SQL často pracujeme s textem – jmény, poznámkami, adresami... A právě pro práci s těmito daty slouží textové funkce. Pomocí nich můžeme získat délku textu, měnit velikost písmen, nahrazovat části textu, vybírat podřetězce, spojovat texty nebo například zjistit, zda se v textu něco nachází.

Textové funkce patří mezi nejčastěji používané funkce vůbec, protože téměř každá databáze obsahuje nějaká textová data.

V následujících příkladech používáme vždy Unicode prefix N před textem, abychom zajistili správné zpracování českých znaků na SQL Serveru.

24.1.1 Délka textu – LEN

Vrací počet znaků ve zvoleném textu. Nezapočítává koncové mezery.

```
-- Výstup: 7  
SELECT LEN(N'Abeceda');
```

```
-- Výstup: 9 (mezery uvnitř textu se počítají)  
SELECT LEN(N' Abe ceda');
```

24.1.2 Spojení více řetězců – CONCAT

Spojuje více textových řetězců dohromady. Velmi vhodné např. pro sestavení celého jména nebo adresy.

```
-- Výstup: Jan Novák  
SELECT CONCAT(N'Jan', ' ', N'Novák');
```

```
-- Spojení sloupců jméno a příjmení  
SELECT CONCAT(Jmeno, ' ', Prijmeni) FROM Zamestnanec;
```

24.1.3 Změna velikosti písmen – LOWER a UPPER

LOWER() převede text na malá písmena, UPPER() na velká písmena.

```
-- Výstup: databáze jsou nej!  
SELECT LOWER(N'Databáze jsou NEJ!');
```

```
-- Výstup: DATABÁZE JSOU NEJ!  
SELECT UPPER(N'Databáze jsou NEJ!');
```

24.1.4 Nahrazení textu – REPLACE

Najde část textu a nahradí ji jiným výrazem.

```
-- Výstup: Mám strojovnu  
SELECT REPLACE(N'Mám mozek', N'mozek', N'strojovnu');
```

```
-- Výstup: Pan Smith  
SELECT REPLACE(N'Mr. Smith', N'Mr.', N'Pan');
```

24.1.5 Výběr části textu – SUBSTRING

Vrátí část textu od určité pozice a o určité délce.

```
-- Výstup: Hello  
SELECT SUBSTRING(N'Hello World', 1, 5);
```

```
-- Výstup: SQL  
SELECT SUBSTRING(N'Databáze SQL Server', 11, 3);
```

24.1.6 Část textu zleva nebo zprava LEFT a RIGHT

Vrací určitý počet znaků zleva (LEFT) nebo zprava (RIGHT).

```
-- Výstup: Databáze  
SELECT LEFT(N'Databáze SQL Server', 8);
```

```
-- Výstup: SQL Server  
SELECT RIGHT(N'Databáze SQL Server', 10);
```

24.1.7 Odstranění mezer – TRIM, LTRIM, RTRIM

Odstraňují mezery na začátku a konci textu. TRIM() odstraní mezery z obou stran, LTRIM() zleva, RTRIM() zprava.

```
-- Výstup: Zdislav
SELECT TRIM(N'   Zdislav  ');
```

```
-- Výstup: Zdislav____
SELECT LTRIM(N'   Zdislav  ');
```

```
-- Výstup: ____Zdislav
SELECT RTRIM(N'   Zdislav  ');
```

24.1.8 Hledání pozice znaku – CHARINDEX

Vrací pozici, kde se hledaný výraz **poprvé** vyskytuje v textu. Pokud nenalezen, vrací 0. Pozor, indexuje se od hodnoty 1, nikoliv od nuly, jak bychom mohli být zvyklí z programování. Pokud je znak na osmé pozici, výsledkem je opravdu číslo 8.

```
-- Výstup: 8
SELECT CHARINDEX(N'SQL', N'Miluji SQL Server');
```

```
-- Výstup: 0 (nenalezeno)
SELECT CHARINDEX(N'MySQL', N'Miluji SQL Server');
```

24.1.9 Obrácení textu – REVERSE

Vrátí text pozpátku.

```
-- Výstup: revreS LQS ezábataD  
SELECT REVERSE(N'Databáze SQL Server');
```

```
-- Výstup: !JEN uosj ezábataD  
SELECT REVERSE(N'Databáze jsou NEJ!');
```

24.2 Numerické funkce

V SQL databázích velmi často pracujeme s čísly – ať už jde o mzdy, ceny, nebo počty položek. Abychom mohli s číselnými hodnotami efektivně počítat, porovnávat je nebo zjišťovat jejich vlastnosti, máme k dispozici sadu numerických funkcí.

Numerické funkce se zapisují stejně jako ostatní – název funkce a v závorkách hodnota nebo sloupec, se kterým chceme pracovat.

24.2.1 Absolutní hodnota – ABS

Vrací kladnou hodnotu zadaného čísla – odstraní případné mínus.

```
-- Výstup: 235  
SELECT ABS(-235);
```

```
-- Výstup: 78  
SELECT ABS(78);
```

24.2.2 Umocnění – POWER

Vrací číslo umocněné na zadaný exponent.

```
-- Výstup: 9 (3 na druhou)
SELECT POWER(3, 2);
```

```
-- Výstup: 1024 (2 na desátou)
SELECT POWER(2, 10);
```

24.2.3 Druhá odmocnina – SQRT

Vrací druhou odmocninu ze zadaného čísla. Funguje pouze na nezáporná čísla.

```
-- Výstup: 3
SELECT SQRT(9);
```

```
-- Výstup: 4.3589...
SELECT SQRT(19);
```

24.2.4 Zaokrouhlení – ROUND

Zaokrouhlí číslo na zadaný počet desetinných míst.

```
-- Výstup: 85.526
SELECT ROUND(85.525644699, 3);
```

```
-- Výstup: 86
SELECT ROUND(85.525644699, 0);
```

24.2.5 Součet – SUM

Vrací součet všech čísel ve sloupci.

```
-- Výstup: součet mezd všech zaměstnanců  
SELECT SUM(Mzda) FROM Zamestnanec;
```

```
-- Součet jen u zaměstnanců z oddělení 1  
SELECT SUM(Mzda) FROM Zamestnanec WHERE OddeleniId = 1;
```

24.2.6 Průměr – AVG

Vrací aritmetický průměr hodnot ve sloupci.

```
-- Výstup: průměrná mzda  
SELECT AVG(Mzda) FROM Zamestnanec;
```

```
-- Průměr jen pro zaměstnance z Ostravy  
SELECT AVG(Mzda) FROM Zamestnanec WHERE Mesto = N'Ostrava';
```

24.2.7 Počet – COUNT

Vrací počet řádků nebo neprázdných hodnot.

```
-- Výstup: počet všech zaměstnanců  
SELECT COUNT(*) FROM Zamestnanec;
```

```
-- Výstup: počet zaměstnanců, kteří mají vyplněnou poznámku  
SELECT COUNT(Poznamka) FROM Zamestnanec;
```



Zřejmě nejčastějším způsobem, jak zaručeně pojmout **všechny řádky tabulky**, je použít ve funkci **hvězdičku**: COUNT(*)).

24.2.8 Nejmenší a největší hodnota – MIN a MAX

Vrací nejmenší, resp. největší hodnotu ve sloupci.

```
-- Výstup: nejnižší mzda  
SELECT MIN(Mzda) FROM Zamestnanec;
```

```
-- Výstup: nejvyšší mzda  
SELECT MAX(Mzda) FROM Zamestnanec;
```

24.3 Funkce pro datum a čas

V SQL databázích je běžné pracovat s daty a časem – například zjistit aktuální datum, spočítat věk uživatele, nebo vybrat záznamy podle měsíce. K tomu slouží funkce pro práci s datovým typem DATE, TIME nebo DATETIME.

24.3.1 Aktuální datum a čas – GETDATE

Vrací aktuální systémové datum a čas ve formátu DATETIME.

```
SELECT GETDATE();  
-- Výstup např.: 2025-07-11 14:23:48.500
```

24.3.2 Získání roku, měsíce, dne – YEAR, MONTH, DAY

Z daného data vrací jeho jednotlivé části.

```
SELECT YEAR('1996-02-06'); -- Výstup: 1996  
SELECT MONTH('1996-02-06'); -- Výstup: 2
```

24.3.3 Část data podle volby – DATEPART

Vrací specifikovanou část data, například týden v roce, den v týdnu atd.

```
SELECT DATEPART(DAY, '2025-07-11');           -- Výstup: 11
SELECT DATEPART(DAYOFYEAR, '2025-07-11');     -- Výstup: 192
SELECT DATEPART(WEEK, GETDATE());             -- Číslo týdne
SELECT DATEPART(WEEKDAY, GETDATE());          -- Den v týdnu (1-7)
SELECT DATEPART(MONTH, '2025-07-11');         -- Výstup: 7
SELECT DATEPART(YEAR, '2025-07-11');          -- Výstup: 2025
SELECT DATEPART(HOUR, '2025-07-11 14:23:00'); -- Výstup: 14
SELECT DATEPART(MINUTE, '2025-07-11 14:23:00'); -- Výstup: 23
SELECT DATEPART(SECOND, '2025-07-11 14:23:05'); -- Výstup: 5
```

24.3.4 Rozdíl mezi daty – DATEDIFF

Spočítá rozdíl mezi dvěma daty ve zvolené jednotce (např. dny, roky).

```
SELECT DATEDIFF(YEAR, '1990-01-01', GETDATE()); -- Věk v letech
SELECT DATEDIFF(DAY, '2025-01-01', '2025-07-11'); -- Počet dní
```

24.3.5 Přičítání k datu – DATEADD

Přičítá zvolený počet jednotek (dní, měsíců, let...) k datu.

```
SELECT DATEADD(DAY, 10, '2025-04-03'); -- Přičte 10 dní
SELECT DATEADD(MONTH, 3, '2025-01-01'); -- Přičte 3 měsíce
```

24.3.6 Převod na typ DATE nebo TIME – ČÁST

Slouží k převodu typu DATETIME na čisté datum nebo čas.

```
SELECT CAST('2025-04-03 10:15:00' AS DATE); -- Výstup: 2025-04-03
SELECT CAST('2025-04-03 10:15:00' AS TIME); -- Výstup: 10:15:00
```

24.3.7 Formátování datumu a času jako text – FORMAT

Vrací datum nebo čas jako řetězec v daném formátu a jazyce.

```
SELECT FORMAT(GETDATE(), 'dd. MMMM yyyy', 'cs-CZ'); -- 11. července 2025
SELECT FORMAT(GETDATE(), 'yyyy/MM/dd');             -- 2025/07/11
```

24.4 Ostatní užitečné funkce

Kromě běžných textových, numerických nebo datumových funkcí nabízí SQL Server i další speciální funkce, které využijeme například pro:

- zjištění aktuální databáze
- určení uživatele
- získání ID posledního vloženého záznamu nebo

24.4.1 Název aktuální databáze – DB_NAME

Funkce vrací název databáze, ve které dotaz právě běží.

```
-- Výstup: např. SkolaDB  
SELECT DB_NAME();
```

24.4.2 ID posledního vloženého záznamu – SCOPE_IDENTITY

Vrací identifikátor posledního vloženého řádku v aktuálním rozsahu (např. po INSERT).

```
INSERT INTO Oddeleni (Nazev) VALUES(N'Nové oddělení');  
SELECT SCOPE_IDENTITY();  
-- Výstup: např. 11
```

24.4.3 Uživatelské jméno – SYSTEM_USER

Vrací aktuálně přihlášeného uživatele (včetně názvu domény nebo počítače).

```
SELECT SYSTEM_USER;  
-- Výstup: např. TELEINFORMATIKA\jan.novak
```

25 Seskupení výsledků – GROUP BY

V mnoha případech nás u výstupu nezajímají jednotlivé řádky, ale spíše souhrnné informace – například kolik zaměstnanců je v každém oddělení, jaký je průměrný plat podle města, nebo jaké jsou celkové tržby podle roku.

K tomu slouží klauzule **GROUP BY**, která nám umožní seskupit řádky podle zvoleného sloupce. Následně můžeme na každou skupinu aplikovat agregační funkce jako **COUNT**, **SUM**, **AVG**, **MIN**, nebo **MAX**.

25.1 Syntaxe

Konstrukce GROUP BY se používá ve spojení s příkazem SELECT a s agregačními funkcemi, abychom seskupili záznamy do logických celků. Syntaxe vypadá následovně:

```
SELECT [sloupce ke skupině], [agregační funkce]
FROM [tabulka]
GROUP BY [sloupce, podle kterých chceme seskupit];
```



Každý sloupec uvedený v SELECT, který není uvnitř agregační funkce, musí být také uveden ve GROUP BY.

Když seskupujeme záznamy, SQL Server vytvoří skupiny řádků podle zvoleného sloupce či sloupců. A v každé skupině může být více hodnot – například více jmen, měst nebo mezd.

Pokud bychom chtěli do výstupu přidat nějaký jiný sloupec, který se v rámci těchto skupin může lišit, databáze neví, jakou hodnotu má z té skupiny zobrazit – a dojde k chybě.

Chybný dotaz:

```
SELECT OddeleniId, Jmeno, COUNT(*)
FROM Zamestnanec
GROUP BY OddeleniId;
```

Správným řešením může být například přidat atribut Jmeno do klauzule GROUP BY

```
SELECT OddeleniId, Jmeno, COUNT(*)  
FROM Zamestnanec  
GROUP BY OddeleniId, Jmeno;
```

25.2 Příklady

Následující dotaz seskupí zaměstnance podle OddeleniId a spočítá, kolik jich je v každém oddělení. Užitečné pro jednoduchý přehled nebo sestavení grafu.

```
SELECT OddeleniId, COUNT(*) AS PocetZamestnancu  
FROM Zamestnanec  
GROUP BY OddeleniId;
```

Další dotaz vypočítá průměrnou mzdu zaměstnanců ve všech městech, kde firma působí. Užitečné pro srovnání životních nákladů nebo plánování rozpočtu.

```
SELECT Mesto, AVG(Mzda) AS PrumernaMzda  
FROM Zamestnanec  
GROUP BY Mesto;
```

Třetí dotaz zjistí, jaká je nejvyšší mzda na každé pracovní pozici. Může pomoci HR oddělení při nastavení mzdových stropů.

```
SELECT Pozice, MAX(Mzda) AS NejvyssiMzda  
FROM Zamestnanec  
GROUP BY Pozice;
```

Následující dotaz ukazuje kombinaci seskupení – tedy kolik zaměstnanců pracuje v každém oddělení v každém městě. Je to složitější struktura, ale velmi užitečná pro geografické rozdělení.

```
SELECT Mesto, OddeleniId, COUNT(*) AS Pocet  
FROM Zamestnanec  
GROUP BY Mesto, OddeleniId;
```

26 Spojování tabulek

V relační databázi jsou informace uloženy ve více tabulkách, které spolu navzájem souvisí. Tento přístup odpovídá principu normalizace – data jsou rozdělena do menších, smysluplně strukturovaných tabulek a propojena pomocí klíčů. Místo abychom uchovávali všechny informace v jedné tabulce, dáváme přednost oddělení údajů a jejich spojování pomocí dotazů.

Například zaměstnanci mohou být evidováni v jedné tabulce a jejich oddělení v jiné. Abychom získali informace o zaměstnanci i jeho oddělení, musíme tyto tabulky propojit. A k tomu slouží právě příkaz **JOIN**, který spojí záznamy z více tabulek podle společného sloupce – obvykle primárního a cizího klíče.

V SQL Serveru existuje několik typů spojení, přičemž nejčastěji se používají vnitřní (**INNER JOIN**) a vnější spojení (**OUTER JOIN**). Vnější spojení se pak dále dělí na levé, pravé a úplné.

26.1 Vnitřní spojení

Vnitřní spojení je nejběžnější typ spojení tabulek v SQL. Pomocí něj vybíráme pouze ty řádky, které mají odpovídající záznam v obou spojených tabulkách.

V praxi to znamená, že spojíme dvě tabulky tak, že hledáme řádky, které mají shodnou hodnotu ve sloupci, který je pro obě tabulky významově propojený – typicky jde o primární a cizí klíč.

```
SELECT sloupce
FROM TabulkaA
INNER JOIN TabulkaB ON TabulkaA.Sloupec = TabulkaB.Sloupec;
```

Místo názvů tabulek často používáme aliasy, s nimiž jsme se už seznámili:

```
SELECT A.Sloupec1, B.Sloupec2
FROM TabulkaA AS A
INNER JOIN TabulkaB AS B ON A.SloupecCK = B.SloupecPK;
```



Název aliasu si můžeme zvolit podle vlastních potřeb. Například u tabulek **Zamestnanec** a **Oddeleni** by to mohly být aliasy **Z** a **O**.

Příklad 1: Zaměstnanci a jejich oddělení

Máme dvě tabulky – **Zamestnanec** a **Oddeleni**. V tabulce Zamestnanec je sloupec OddeleniId, který slouží jako cizí klíč a odkazuje na primární klíč Id v tabulce Oddeleni. Pomocí INNER JOIN vybíráme jen ty zaměstnance, kteří mají existující přiřazené oddělení, tedy jejich OddeleniId odpovídá nějakému Id v tabulce oddělení.

Ve výsledku dostaneme jména zaměstnanců a názvy oddělení, ale **nezobrazí** se žádný zaměstnanec, který nemá oddělení přiřazeno (např. OddeleniId = NULL nebo odkaz na neexistující Id).

```
SELECT Z.Jmeno, O.Nazev AS Oddeleni
FROM Zamestnanec AS Z
INNER JOIN Oddeleni AS O ON Z.OddeleniId = O.Id;
```

Příklad 2: Produkty a jejich kategorie

Zde spojujeme tabulku **Produkt**, kde každý produkt patří do určité kategorie, s tabulkou **Kategorie**, která obsahuje názvy jednotlivých kategorií. Spojení probíhá přes sloupce P.KategorieId a K.Id.

Dotaz vyhledá všechny produkty, které mají přiřazenou nějakou kategorii, a vypíše jejich název spolu s názvem této kategorie. Produkty bez přiřazené nebo neexistující kategorie se ve výsledku vůbec neobjeví, jsou totiž vyloučeny vnitřním spojením.

```
SELECT P.Nazev AS Produkt, K.Nazev AS Kategorie
FROM Produkt AS P
INNER JOIN Kategorie AS K ON P.KategorieId = K.Id;
```

Příklad 3: Studenti a jejich třídy

Každý student v tabulce **Student** je zařazen do třídy, kterou určuje cizí klíč TridaId. Tabulka **Trida** obsahuje informace o jednotlivých třídách, přičemž každý řádek má svůj jedinečný identifikátor Id.

Dotaz pomocí INNER JOIN spojuje tyto dvě tabulky přes S.TridaId = T.Id, čímž získáme seznam studentů doplněný o název jejich třídy.

Opět platí, že studenti bez přiřazené třídy nebo s nesprávně vyplněným TridaId se v dotazu neobjeví – protože INNER JOIN zobrazí pouze záznamy, kde existuje odpovídající dvojice.

```
SELECT S.Jmeno, T.Nazev AS Trida
FROM Student AS S
INNER JOIN Trida AS T ON S.TridaId = T.Id;
```

26.2 Vnější spojení

V praxi se často setkáme se situací, kdy potřebujeme pracovat i s nekompletními daty – tedy s řádky, které nemají odpovídající protějšek v jiné tabulce. Například chceme zobrazit všechny zaměstnance, i ty, kteří ještě nejsou přiřazeni k žádnému oddělení, nebo všechny produkty v katalogu, včetně těch, které nepatří do žádné kategorie.

Pro tyto případy nestačí běžné vnitřní spojení (INNER JOIN), protože to zobrazí jen řádky s odpovídajícími hodnotami v obou tabulkách. V takových situacích použijeme **vnější spojení** (OUTER JOIN), které je navrženo tak, aby zachovalo všechny řádky z jedné (nebo obou) tabulek, a tam, kde chybí odpovídající záznam, doplní místo něj NULL.

26.2.1 LEFT JOIN

Levé vnější spojení LEFT OUTER JOIN (zkráceně často jen LEFT JOIN) slouží k tomu, abychom ve výsledku zobrazili všechny řádky z levé tabulky, a to i v případě, že k nim v pravé tabulce neexistuje žádný odpovídající záznam. Pokud odpovídající řádek v pravé tabulce existuje, spojí se s levým. Pokud ne, do pravé části se automaticky doplní NULL.

Tento typ spojení se často používá například tehdy, když chceme vidět všechny zákazníky, včetně těch, kteří ještě neprovedli žádnou objednávku.



Pojem „levá tabulka“ v SQL znamená tu tabulku, která se nachází **vlevo od klíčového slova JOIN** v SQL dotazu. Tedy ne vlevo fyzicky v databázi, ale v levé části zápisu dotazu.

Například:

```
-- Zde je levou tabulkou Zakaznik, protože je uvedena před LEFT JOIN.
SELECT *
FROM Zakaznik
LEFT JOIN Objednavka ON Zakaznik.Id = Objednavka.ZakaznikId;
```

Příklad 1: Zákazníci a jejich objednávky (i ti bez objednávky)

Zákazník je zde levá tabulka (Zakaznik je uveden vlevo od LEFT JOIN). Dotaz vypíše všechny zákazníky, včetně těch, kteří zatím žádnou objednávku neprovedli.

Pokud zákazník má objednávky, zobrazí se jejich čísla a data. Pokud nemá žádnou, zobrazí se jeho jméno, ale ostatní sloupce budou NULL.

```
SELECT Z.Jmeno, O.CisloObjednavky, O.Datum
FROM Zakaznik AS Z
LEFT JOIN Objednavka AS O ON Z.Id = O.ZakaznikId;
```

Příklad 2: Učitelé a jejich přiřazené předměty (i bez výuky)

Levou tabulkou je Ucitel, takže dotaz zobrazí všechny učitele, i ty, kterým není přiřazen žádný předmět k výuce. U těchto učitelů bude sloupec Predmet prázdný (NULL), ale jméno učitele se ve výsledku zobrazí.

```
SELECT U.Jmeno, P.Nazev AS Predmet
FROM Ucitel AS U
LEFT JOIN Predmet AS P ON U.Id = P.UcitelId;
```

Příklad 3: Produkty a jejich výrobci (i bez výrobce)

Zobrazíme seznam všech produktů, včetně těch, které nemají uvedeného žádného výrobce. Díky LEFT JOIN se do výsledku dostane každý produkt, a pokud má přiřazeného výrobce, zobrazí se jeho název. Pokud ne, pole Vyrobce bude NULL.

```
SELECT P.Nazev, V.Nazev AS Vyrobce
FROM Produkt AS P
LEFT JOIN Vyrobce AS V ON P.VyrobceId = V.Id;
```

26.2.2 RIGHT JOIN

Pravé vnější spojení (RIGHT JOIN) funguje obdobně jako levé spojení – s tím rozdílem, že zachovává všechny řádky z pravé tabulky, i pokud k nim neexistuje žádný odpovídající záznam v tabulce levé.

To znamená, že i nepřirazené záznamy v pravé tabulce budou ve výsledku vidět – a tam, kde k nim chybí odpovídající data v levé tabulce, se do těchto míst doplní NULL.

Příklad 1: Všechny předměty, i když nemají učitele

Tento dotaz zobrazí všechny předměty, i ty, které nemají přiřazeného učitele.

Pravá tabulka je Predmet, a proto jsou její řádky vždy zahrnuty ve výstupu. U předmětů bez učitele bude sloupec Ucitel obsahovat NULL.

```
SELECT U.Jmeno AS Ucitel, P.Nazev AS Predmet
FROM Ucitel AS U
RIGHT JOIN Predmet AS P ON U.Id = P.UcitelId;
```

Příklad 2: Oddělení a jejich vedoucí (i bez vedoucího)

Chceme vidět všechna oddělení, bez ohledu na to, zda mají přiřazeného vedoucího.

Pravá tabulka je Oddeleni, a ta bude ve výstupu kompletní. Pokud oddělení nemá žádného vedoucího, hodnota ve sloupci Vedouci bude NULL.

```
SELECT V.Jmeno AS Vedouci, O.Nazev AS Oddeleni
FROM Vedouci AS V
RIGHT JOIN Oddeleni AS O ON V.OddeleniId = O.Id;
```

Příklad 3: Všechny role ve firmě a přiřazení zaměstnanci

Zobrazujeme všechny role, včetně těch, ke kterým není zatím přiřazen žádný zaměstnanec.

Pravá tabulka je Role, takže se všechny její záznamy objeví ve výsledku. U těch rolí, ke kterým zatím nikdo nepatří, bude sloupec Zamestnanec prázdný (NULL).

```
SELECT Z.Jmeno AS Zamestnanec, R.Nazev AS Role
FROM Zamestnanec AS Z
RIGHT JOIN Role AS R ON Z.RoleId = R.Id;
```

26.2.3 FULL JOIN

FULL OUTER JOIN (zkráceně FULL JOIN) je nejkomplexnější z vnějších spojení. Na rozdíl od LEFT a RIGHT JOIN, které zachovávají řádky pouze z jedné strany, FULL JOIN **zachovává všechny řádky z obou tabulek** – a tam, kde chybí odpovídající protějšek, dosadí NULL.

Jinými slovy, FULL JOIN vytvoří sjednocení levého a pravého spojení. Používá se v situacích, kdy potřebujeme kompletní pohled na data z obou tabulek, včetně těch, která se nepárují.

Například:

```
SELECT sloupce
FROM Tabulka1
FULL JOIN Tabulka2 ON Tabulka1.Sloupec = Tabulka2.Sloupec;
```

Pokud existuje odpovídající záznam v obou tabulkách, spojí se dohromady. Pokud existuje pouze v jedné z tabulek, ve sloupcích z druhé tabulky se zobrazí NULL.

Příklad 1: Zákazníci a objednávky (i neaktivní i sirotci)

Uvažujme situaci, kdy máme tabulku Zakaznik a tabulku Objednavka. Pomocí následujícího dotazu získáme výpis všech zákazníků a všech objednávek. Pokud má zákazník přiřazenou objednávku, spojí se jejich údaje do jednoho řádku. Pokud zákazník žádnou objednávku nemá, přesto se ve výstupu objeví, ale informace o objednávce budou NULL. Stejně tak, pokud je v tabulce Objednavka řádek, který se neváže k žádnému zákazníkovi (například kvůli chybě), ve výstupu se také objeví, ale bez údajů o zákazníkovi.

```
SELECT Z.Jmeno AS Zakaznik, O.CisloObjednavky
FROM Zakaznik AS Z
FULL JOIN Objednavka AS O ON Z.Id = O.ZakaznikId;
```

Příklad 2: Studenti a zápisy ke zkouškám

Představme si tabulku Student, kde jsou evidováni všichni studenti, a tabulku Zapis, kde se zaznamenává, kdo je přihlášen ke které zkoušce. Dotaz vrátí seznam všech studentů a všech zápisů ke zkouškám. Studenti, kteří jsou přihlášení, se zobrazí spolu s názvem zkoušky. Ti, kteří se zatím k žádné zkoušce nepřihlásili, se také objeví, ale sloupec NazevZkousky bude prázdný (NULL). A naopak – pokud by záznam o zápisu ke zkoušce neodpovídal žádnému existujícímu studentovi (například kvůli chybnému identifikátoru), tak se i tento záznam zobrazí, ale bez informací o studentovi.

```
SELECT S.Jmeno, Z.NazevZkousky
FROM Student AS S
FULL JOIN Zapis AS Z ON S.Id = Z.StudentId;
```

Příklad 3: Produkty a jejich dodavatelé

Ve třetím příkladu spojíme tabulku Produkt s tabulkou Dodavatel, přičemž spojení provádíme na základě identifikátoru dodavatele. Tento dotaz vrátí seznam všech produktů i všech dodavatelů. Produkty, které mají přiřazeného dodavatele, budou zobrazeny spolu s jeho názvem. Produkty bez dodavatele budou mít ve sloupci Dodavatel hodnotu NULL. A naopak – i dodavatelé, kteří aktuálně nejsou přiřazeni k žádnému produktu, se ve výsledku objeví. Jejich jméno bude uvedeno, ale informace o produktu bude chybět.

```
SELECT P.Nazev AS Produkt, D.Nazev AS Dodavatel
FROM Produkt AS P
FULL JOIN Dodavatel AS D ON P.DodavatelId = D.Id;
```



Je důležité si uvědomit, že pokud **jeden řádek z jedné tabulky odpovídá více řádkům z druhé tabulky**, bude se tento řádek ve výstupu objevovat **opakovaně – tolikrát, kolik má odpovídajících záznamů**.

Například pokud má jeden zákazník více objednávek, a použijeme spojení přes **JOIN** (INNER, LEFT, RIGHT i FULL), bude se tento zákazník ve výsledku opakovat u každé své objednávky.

27 Oprávnění

V každé databázi je nezbytné zajistit, aby k datům a funkcím systému měli přístup jen ti, kteří jej skutečně potřebují. SQL Server k tomu poskytuje propracovaný systém řízení přístupu pomocí přihlašovacích údajů (logins), uživatelů (users) a oprávnění (permissions), která lze udělit (GRANT) nebo odebrat (REVOKE).

V této kapitole si ukážeme základní principy toho, jak SQL Server přistupuje k zabezpečení přístupu, jak vytvořit přihlašovací údaje a uživatele v databázi a jak těmto uživatelům přidělit (nebo odebrat) konkrétní oprávnění. Tato témata jsou zásadní především při vývoji víceuživatelských systémů, kde je potřeba jasně omezit, kdo může číst, měnit nebo mazat data.

27.1 LOGIN

Login představuje identitu, která se může přihlásit k SQL Serveru jako celku. Login je definován na úrovni serveru a umožňuje uživateli navázat spojení se serverem, nikoli však automaticky přístup ke konkrétním databázím.

Existují dva hlavní typy loginů:

- **SQL Server login** – přihlašování pomocí uživatelského jména a hesla uloženého v SQL Serveru.
- **Windows login** – přihlašování pomocí uživatelského účtu ze systému Windows. Tento účet může pocházet ze serveru Active Directory, ale také se může jednat o lokální uživatelský účet přímo z počítače, na kterém běží SQL Server.

Příklad vytvoření loginu pomocí SQL Server autentizace:

```
CREATE LOGIN UzivatelJana WITH PASSWORD = N'TajneHeslo123';
```

Příklad vytvoření loginu pomocí lokálního Windows účtu:

```
CREATE LOGIN [PC-JAN\Jan] FROM WINDOWS;
```

Příklad vytvoření loginu pomocí Windows Active Directory účtu:

```
CREATE LOGIN [TELEINFORMATIKA\jan.novak] FROM WINDOWS;
```

27.2 USER

User představuje identitu v rámci konkrétní databáze. Každý login může mít v jedné nebo více databázích přiřazeného uživatele, čímž získá přístup k jejím datům a objektům.

Uživatele vždy přiřazujeme k existujícímu loginu. Bez vytvoření USER v databázi by login neměl k této databázi přístup, i když se umí přihlásit na server.

```
USE MojeDatabaze;  
CREATE USER JanNovak FOR LOGIN [TELEINFORMATIKA\jan.novak];
```



Protože USER je nastavován pro konkrétní databázi, je vždy potřeba tento příkaz vztahovat k některé z nich. Všimněme si, že v kódu začínáme nejprve příkazem **USE MojeDatabaze**;

Tím SQL Serveru říkáme, které databáze se bude následující příkaz týkat.

27.3 GRANT

Pomocí příkazu GRANT udělujeme uživateli nebo roli právo vykonávat určitou akci nad objekty databáze, například číst data, vkládat nové záznamy nebo mazat tabulky.

Zde jsou některá z nejčastěji používaných oprávnění, která lze pomocí GRANT udělit:

Oprávnění	Význam
SELECT	Povolení čtení dat z tabulek
INSERT	Povolení vkládání nových řádků do tabulek
UPDATE	Povolení upravovat existující data v tabulce
DELETE	Povolení mazat záznamy z tabulky
ALTER	Povolení měnit strukturu tabulky (např. přidávat sloupce)
CREATE	Povolení vytvářet nové objekty (např. tabulky)
DROP	Povolení mazat objekty (např. tabulky)
EXECUTE	Povolení spouštět uložené procedury a funkce

27.3.1 Udělení jednoho oprávnění

Chceme-li udělit jedno konkrétní oprávnění (např. čtení dat), použijeme příkaz GRANT s jedním typem oprávnění. Uživatel tak získá přístup jen k dané operaci. V tomto případě uživateli Jana povolíme čtení (SELECT) z tabulky Zamestnanec, která je součástí schématu dbo. Uživatel může zobrazovat data, ale nesmí je měnit, mazat ani přidávat nové.

```
USE Firma;  
GRANT SELECT ON dbo.Zamestnanec TO Jana;
```

27.3.2 Udělení více oprávnění najednou

Chceme-li jednomu uživateli udělit více práv současně, uvedeme je jako seznam oddělený čárkami. Toto řešení je přehledné a výhodné, pokud uživatel potřebuje např. plný přístup k datům v rámci tabulky.

Následujícím příkazem získá Jana možnost číst, vkládat, upravovat i mazat data v tabulce Zamestnanec. Přestože jde o jednu tabulku, díky více oprávněním má uživatel v rukou kompletní sadu operací, což může být vhodné např. pro administrátory nebo vedoucí pracovníky.

```
USE Firma;  
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Zamestnanec TO Jana;
```

27.3.3 Udělení oprávnění na konkrétní sloupce

Ve speciálních případech může být potřeba povolit uživateli přístup jen k určitým sloupcům tabulky – např. pokud má mít přístup pouze ke jménu a příjmení zaměstnanců, ale ne k jejich platům či rodným číslům.

Následujícím příkazem uživateli Jana povolíme číst pouze sloupce Jmeno a Prijmeni. Jakýkoli pokus o přístup ke sloupci Plat nebo jiným sloupcům bude SQL Serverem odmítnut jako nedovolený přístup.

Tato forma oprávnění je velmi užitečná například u tabulek, které obsahují citlivé nebo osobní údaje, kde je nutné omezit přístup jen na nezbytné informace.

```
USE Firma;  
GRANT SELECT (Jmeno, Prijmeni) ON dbo.Zamestnanec TO Jana;
```



Při přidělování oprávnění bychom se měli vždy řídit **zásadou minimálního oprávnění**, což je bezpečnostní princip, podle kterého by každý uživatel měl mít přístup pouze k těm datům a operacím, které nezbytně potřebuje ke své práci – a **nic navíc**. Smyslem je omezit riziko chyb, zneužití nebo nechtěného poškození dat, a zároveň zpřehlednit správu oprávnění v systému.

Tento přístup platí nejen v databázích, ale obecně ve všech systémech s řízeným přístupem. Udělujeme raději méně práv a postupně přidáváme podle potřeby, než abychom rovnou poskytli úplný přístup všem uživatelům.

27.4 REVOKE

Příkaz REVOKE slouží k odebrání dříve udělených oprávnění uživateli. Pokud již daný uživatel nemá oprávnění pro určitou operaci (například čtení tabulky), SQL Server mu tuto akci nedovolí provést – dojde k chybě oprávnění.

Pomocí REVOKE můžeme odebrat oprávnění jednotlivě, hromadně nebo také pouze pro konkrétní sloupce. Tento příkaz je tedy protikladem GRANT, ale pracuje se stejnou strukturou a syntaxí.

27.4.1 Odebrání jednoho oprávnění

Tímto příkazem uživateli Jana odebíráme právo číst z tabulky Zamestnanec v databázi Firma. Jakýkoli pokus o čtení z této tabulky skončí chybou oprávnění.

```
USE Firma;  
REVOKE SELECT ON dbo.Zamestnanec FROM Jana;
```

27.4.2 Odebrání více oprávnění najednou

Tento příkaz najednou odebere dvě oprávnění – vkládání (INSERT) a úpravy (UPDATE). Uživatel sice může mít i další oprávnění (například SELECT), ale ta tímto zůstanou zachována.

```
USE Firma;  
REVOKE INSERT, UPDATE ON dbo.Zamestnanec FROM Jana;
```

27.4.3 Odebrání oprávnění na konkrétní sloupce

Zde odebíráme možnost číst konkrétní sloupce tabulky. Pokud měl uživatel Jana dříve povoleno číst např. Jmeno, tímto příkazem už o toto právo přijde a pokus o výběr těchto sloupců bude zamítnut.

```
USE Firma;  
REVOKE SELECT (Jmeno, Prijmeni) ON dbo.Zamestnanec FROM Jana;
```

27.5 Odebrání uživatele a přihlášení

Jakmile už daný uživatel nemá mít přístup k databázi nebo SQL Serveru, je vhodné ho úplně odstranit. K tomu slouží příkazy **DROP USER** a **DROP LOGIN**.

- **DROP USER** odstraní uživatele pouze z konkrétní databáze.
- **DROP LOGIN** odstraní přihlašovací účet ze serveru – tedy úplně zamezí přihlášení k serveru.

27.5.1 Odstranění uživatele z databáze

Tímto příkazem odstraníme uživatele Jana z databáze Firma. Od té chvíle nebude moci v této databázi provádět žádné akce, ani pokud má stále login na server.

```
USE Firma;  
DROP USER Jana;
```

27.5.2 Odstranění loginu ze serveru

Tímto příkazem zcela odstraníme přihlašovací účet Jana ze serveru SQL. Už se nebude moci vůbec přihlásit, ani k žádné databázi, ke které dříve měla přístup. Protože se login váže k celému serveru, nikoliv ke konkrétní databázi, označení databáze pomocí **USE Firma;** by zde bylo zbytečné.

```
DROP LOGIN Jana;
```



Pokud chceme odstranit login, je nutné **nejprve odstranit uživatele** ve všech databázích, kde byl vytvořen. Jinak příkaz **DROP LOGIN** skončí chybou kvůli existujícím vazbám.